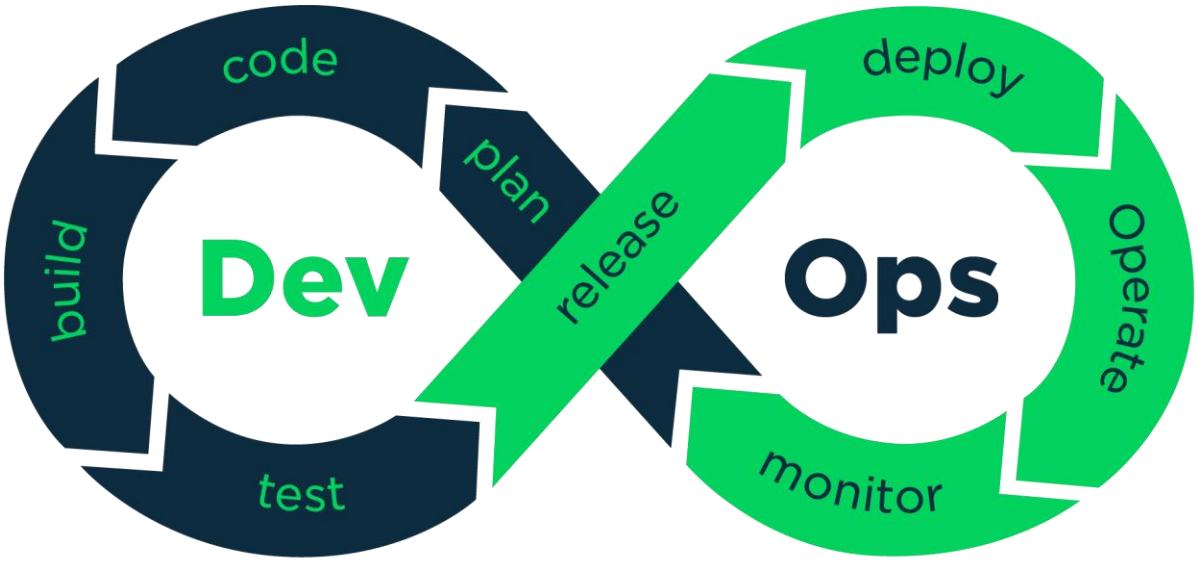
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING AND CYBER SECURITY

SE202L – DEVELOPMENT OPERATIONS LAB MANUAL (SPRING'26)



by
Memoona Amjad

Contents

Lab 1: Introduction to DevOps, OS & Linux Fundamentals 9

Lab Overview 9

Learning Objectives 9

Part 1: Introduction to DevOps 9

1.1 What is DevOps? 9

1.2 DevOps and the Software Development Life Cycle (SDLC) 10

1.3 DevOps Tools Landscape 10

Part 2: Introduction to Operating Systems 11

2.1 What is an Operating System? 11

2.2 Types of Operating Systems 11

2.3 Why Linux for DevOps? 11

Part 3: Linux Filesystem Structure 12

3.1 Understanding the Linux Filesystem Hierarchy 12

3.2 Absolute vs Relative Paths 13

Part 4: Setting Up Your Linux Environment 13

4.1 Accessing Linux 13

4.2 Opening the Terminal 13

Part 5: Basic Linux Commands 14

5.1 Navigation Commands 14

5.2 File and Directory Operations 15

5.3 Viewing File Contents 16

5.4 Getting Help 18

Part 6: Practical Exercises 18

Exercise 1: Exploring the Filesystem 18

Exercise 2: Creating a Directory Structure 19

Exercise 3: Working with Files 19

Exercise 4: File Content Operations 19

Exercise 5: Advanced Navigation 19

Part 7: Understanding Command Structure 20

7.1 Anatomy of a Linux Command 20

7.2 Wildcards and Pattern Matching 20

7.3 Command Chaining 20

Part 8: Common Mistakes and Troubleshooting 21
8.1 Case Sensitivity 21
8.2 Spaces in Filenames 21
8.3 Hidden Files 21
8.4 Permission Denied Errors 21
8.5 Command Not Found 21
Lab Assessment Tasks 22
Lab 2: Linux Files, Permissions & Process Management 23
Lab Overview 23
Learning Objectives 23
Part 1: Advanced File Operations 23
1.1 Finding Files and Directories 23
1.2 Searching Inside Files 24
1.3 File Comparison 25
1.4 File Compression and Archiving 25
Part 2: Understanding File Permissions 26
2.1 Linux Permission Model 26
2.2 Viewing Permissions 26
2.3 Numeric (Octal) Representation 27
2.4 Changing Permissions with chmod 27
2.5 Changing Ownership 28
2.6 Special Permissions 28
Part 3: User and Group Management 29
3.1 Understanding Users and Groups 29
3.2 User Information Commands 29
3.3 Creating and Managing Users 30
3.4 Creating and Managing Groups 30
3.5 Switching Users 31
Part 4: Process Management 31
4.1 Understanding Processes 31
4.2 Viewing Processes 32
4.3 Process States 33
4.4 Managing Processes 33

4.5 Background and Foreground Jobs 34
4.6 Process Priority 34
Part 5: System Resource Management 35
5.1 Disk Usage 35
5.2 Memory Usage 35
5.3 System Information 36
5.4 Monitoring System Logs 36
Part 6: Practical Exercises 37
Exercise 1: File Permissions Practice 37
Exercise 2: User and Group Management 37
Exercise 3: Process Management 38
Exercise 4: Finding and Searching Files 38
Exercise 5: System Monitoring 38
Exercise 6: File Compression 39
Lab Assessment Tasks 39
Lab 3: Shell Scripting, Git Basics & Application Concepts 41
Lab Overview 41
Learning Objectives 41
Part 1: Introduction to Shell Scripting 41
1.1 What is Shell Scripting? 41
1.2 Creating Your First Script 42
1.3 Variables in Bash 42
1.4 Conditional Statements 43
1.5 Loops in Bash 45
1.6 Functions in Bash 47
1.7 Script Arguments 47
Part 2: Git Basics 48
2.1 What is Git? 48
2.2 Installing and Configuring Git 48
2.3 Creating a Git Repository 49
2.4 Basic Git Workflow 49
2.5 Git Status and Tracking 50
2.6 Staging Changes 50

2.7 Making Commits 51
2.8 Viewing History 51
2.9 Viewing Changes 52
2.10 Ignoring Files 52
Part 3: Application Concepts 53
3.1 Understanding Applications 53
3.2 Client-Server Architecture 53
3.3 Frontend vs Backend 54
3.4 HTTP Basics 55
3.5 Introduction to APIs 55
Part 4: Practical Exercises 55
Exercise 1: Basic Shell Scripts 56
Exercise 2: Conditional Logic 57
Exercise 3: Loops 58
Exercise 4: Git Practice 59
Exercise 5: Combining Scripts and Git 61
Part 5: Practical Application Scenarios 62
Scenario 1: Automated Log Cleanup 62
Scenario 2: User Management Script 63
Lab Assessment Tasks 64
Task 1: Personal Information Script 64
Task 2: File Organizer Script 65
Task 3: Git Project Setup 65
Task 4: System Health Check Script 65
Task 5: Git Workflow Documentation 65
Lab 4: Frontend & Backend Fundamentals 67
Lab Overview 67
Learning Objectives 67
Part 1: Frontend Basics - HTML 67
1.1 Introduction to HTML 67
1.2 Common HTML Elements 68
1.3 HTML Forms 69
Part 2: Frontend Styling - CSS 70

2.1 Introduction to CSS 70
2.2 CSS Selectors 71
2.3 Common CSS Properties 72
2.4 Creating a Styled Webpage 73
Part 3: Frontend Interactivity - JavaScript 75
3.1 Introduction to JavaScript 75
3.2 JavaScript Basics 75
3.3 DOM Manipulation 77
3.4 Interactive Example 78
Part 4: Backend Development with Python Flask 81
4.1 Introduction to Flask 81
4.2 Your First Flask Application 82
4.3 Flask Routes and Methods 83
4.4 Building a Complete API 83
4.5 Testing the API with curl 85
Part 5: Alternative Backend - Node.js Express 86
5.1 Introduction to Node.js and Express 86
5.2 Creating an Express Application 86
5.3 Building a Complete API with Express 87
Part 6: Connecting Frontend and Backend 89
6.1 CORS (Cross-Origin Resource Sharing) 89
6.2 Complete Frontend-Backend Example 90
Part 7: Practical Exercises 95
Exercise 1: Create a Personal Portfolio Page 95
Exercise 2: Build a Calculator API 95
Exercise 3: Student Management System 95
Exercise 4: Weather Dashboard 95
Lab Assessment Tasks 96
Task 1: Personal Bio Page 96
Task 2: REST API for Books 96
Task 3: Interactive Quiz Page 96
Task 4: Todo List with Backend 96
Task 5: Documentation 97

Lab 5: Git Collaboration & GitHub 98
Lab Overview 98
Learning Objectives 98
Part 1: Introduction to GitHub 98
1.1 What is GitHub? 98
1.2 Creating a GitHub Account 99
1.3 SSH Key Setup (Recommended) 99
Part 2: Working with Remote Repositories 100
2.1 Creating a Repository on GitHub 100
2.2 Connecting Local and Remote Repositories 100
2.3 Basic Remote Operations 101
Part 3: Branching Strategies 102
3.1 Understanding Branches 102
3.2 Creating and Managing Branches 102
3.3 Common Branching Strategies 104
Part 4: Pull Requests 105
4.1 What is a Pull Request? 105
4.2 Creating a Pull Request 106
4.3 Code Review Process 106
4.4 Merging Pull Requests 107
Part 5: Merge Conflicts 108
5.1 Understanding Merge Conflicts 108
5.2 Creating a Conflict (Practice) 108
5.3 Resolving Conflicts 109
5.4 Aborting a Merge 110
5.5 Preventing Conflicts 111
Part 6: Documentation Best Practices 111
6.1 Writing a Good README 111
6.2 Other Important Documentation Files 113
Part 7: Practical Exercises 114
Exercise 1: First GitHub Repository 115
Exercise 2: Feature Branch Workflow 115
Exercise 3: Collaborative Workflow (Pair Exercise) 117

Exercise 4: Merge Conflict Resolution 118
Exercise 5: GitHub Repository Setup 119
Part 8: Advanced Git Topics 119
8.1 Git Tags 119
8.2 Git Stash 120
8.3 Viewing History 121
8.4 Undoing Changes 122
Lab Assessment Tasks 122
Task 1: Repository Setup 122
Task 2: Feature Development 123
Task 3: Collaboration Simulation 123
Task 4: Conflict Resolution 123
Task 5: Complete Workflow 124

Lab 1: Introduction to DevOps, OS & Linux Fundamentals

Lab Overview

This lab introduces you to the foundational concepts of DevOps, operating systems, and Linux. You will learn what DevOps is, why it matters in modern software development, and how Linux serves as the backbone of most DevOps environments. By the end of this lab, you will be comfortable navigating the Linux filesystem and executing basic terminal commands.

Topics Covered:

- Overview of DevOps and its role in SDLC
- Introduction to Operating Systems
- Linux Filesystem Structure
- Basic Terminal Commands and Navigation

Learning Objectives

By the end of this lab, you will be able to:

1. Explain what DevOps is and its importance in software development
2. Understand the role of operating systems in computing
3. Navigate the Linux filesystem using command-line interface
4. Execute basic Linux commands for file and directory operations
5. Use essential terminal utilities for system exploration

Part 1: Introduction to DevOps

1.1 What is DevOps?

DevOps is a culture, set of practices, and collection of tools that aims to bridge the gap between software development (Dev) and IT operations (Ops). Traditionally, developers would write code and "throw it over the wall" to operations teams who would then deploy and maintain it. This created delays, miscommunication, and reliability issues.

DevOps combines development and operations into a collaborative workflow where:

- Developers understand deployment and infrastructure
- Operations teams are involved early in the development process
- Automation reduces manual errors and speeds up delivery
- Continuous feedback improves quality

Key Benefits of DevOps:

- Faster time to market for new features
- Improved collaboration between teams
- Higher quality and more reliable software
- Automated and repeatable processes
- Rapid recovery from failures

1.2 DevOps and the Software Development Life Cycle (SDLC)

The traditional SDLC follows these phases: Planning, Analysis, Design, Implementation, Testing, Deployment, and Maintenance. In traditional models like Waterfall, these phases happen sequentially, often taking months or years.

DevOps transforms the SDLC by introducing:

Continuous Integration (CI): Developers frequently merge code changes into a shared repository. Automated tests run on every change to catch bugs early.

Continuous Delivery (CD): Code is always in a deployable state. Releases can happen at any time with minimal manual intervention.

Continuous Deployment: Every code change that passes automated tests is automatically deployed to production.

Infrastructure as Code (IaC): Infrastructure is managed using code and configuration files, making it version-controlled and reproducible.

Monitoring and Feedback: Continuous monitoring of applications provides feedback to developers, closing the loop.

This creates a cycle: Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Plan (and repeat).

1.3 DevOps Tools Landscape

Throughout this course, you will work with various DevOps tools:

- **Version Control:** Git, GitHub
- **Containerization:** Docker
- **CI/CD:** GitHub Actions
- **Cloud Infrastructure:** AWS EC2
- **Infrastructure as Code:** Terraform
- **Operating System:** Linux

Each tool serves a specific purpose in the DevOps pipeline, and you will gain hands-on experience with all of them.

Part 2: Introduction to Operating Systems

2.1 What is an Operating System?

An Operating System (OS) is system software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between users/applications and the computer hardware.

Key Functions of an OS:

- **Process Management:** Creating, scheduling, and terminating processes
- **Memory Management:** Allocating and deallocating memory to programs
- **File System Management:** Organizing and storing files on storage devices
- **Device Management:** Managing input/output devices
- **Security and Access Control:** Protecting resources from unauthorized access
- **User Interface:** Providing a way for users to interact with the computer

2.2 Types of Operating Systems

Single-User, Single-Tasking: Can run one program at a time (e.g., MS-DOS)

Single-User, Multi-Tasking: One user can run multiple programs simultaneously (e.g., Windows, macOS)

Multi-User: Multiple users can use the system simultaneously (e.g., Linux, Unix)

Real-Time OS: Designed for time-critical applications (e.g., embedded systems)

Distributed OS: Manages a group of independent computers and makes them appear as a single system

2.3 Why Linux for DevOps?

Linux is the dominant operating system in DevOps and server environments for several reasons:

Open Source: Free to use, modify, and distribute. This reduces costs and provides transparency.

Stability and Reliability: Linux servers can run for years without rebooting. Critical for production systems.

Security: Strong permission model and active security community. Regular updates and patches.

Performance: Efficient resource usage, especially important for servers running multiple services.

Community and Support: Massive global community provides extensive documentation and support.

Flexibility: Highly customizable for specific needs. Can run on anything from tiny embedded devices to massive supercomputers.

Industry Standard: Most cloud servers, containers, and DevOps tools run on Linux. Learning Linux is essential for DevOps professionals.

Part 3: Linux Filesystem Structure

3.1 Understanding the Linux Filesystem Hierarchy

Unlike Windows which uses drive letters (C:, D:, etc.), Linux uses a single unified directory tree starting from the root directory denoted by a forward slash `/`. Everything in Linux is considered a file, including directories, devices, and processes.

Key Directories in Linux:

/ (Root): The top-level directory. All other directories branch from here.

/home: Contains home directories for all users. Each user has a subdirectory here (e.g., /home/username). This is where users store their personal files and configurations.

/root: The home directory for the root (administrator) user. Separate from /home.

/bin: Contains essential user command binaries (programs) that are needed in single-user mode and to bring the system up or repair it (e.g., ls, cp, cat).

/sbin: Contains system binaries, typically used by the system administrator (e.g., fdisk, ifconfig).

/usr: Contains user programs and data. Has its own bin, sbin, and lib subdirectories with additional programs and libraries.

/etc: Contains system-wide configuration files. Almost every program has configuration files here.

/var: Contains variable data like logs, databases, email, and print queues. Data that changes frequently during system operation.

/tmp: Temporary files created by programs. Usually cleared on reboot.

/dev: Contains device files representing hardware devices and virtual devices.

/lib: Contains shared library files needed by programs in /bin and /sbin.

/opt: Contains optional software packages, often third-party applications.

/mnt and /media: Mount points for temporary filesystems and removable media (USB drives, CDs).

3.2 Absolute vs Relative Paths

Absolute Path: Specifies the complete path from the root directory. Always starts with `/`.
Example: `/home/student/documents/file.txt`

Relative Path: Specifies the path relative to the current directory. Does not start with `/`.
Example: If you are in `/home/student`, the relative path to the same file would be `documents/file.txt`

Special Directory Symbols:

- `.` (single dot) represents the current directory
- `..` (double dot) represents the parent directory
- `~` (tilde) represents the current user's home directory
- `/` (forward slash) is the path separator and root directory

Part 4: Setting Up Your Linux Environment

4.1 Accessing Linux

For this lab, you can use one of the following methods:

Option 1: University Lab Computers - If Linux is already installed, simply boot into Linux or access it through dual-boot.

Option 2: Virtual Machine - Install VirtualBox or VMware and create an Ubuntu virtual machine.

Option 3: WSL (Windows Subsystem for Linux) - If you are using Windows 10/11, install WSL2 with Ubuntu.

Option 4: Live USB - Boot Ubuntu from a USB drive without installation.

Your instructor will guide you on the recommended method for this course.

4.2 Opening the Terminal

The terminal (also called shell or command line) is your primary interface for working with Linux in DevOps.

To open the terminal in Ubuntu:

- Press `Ctrl + Alt + T`
- Or search for "Terminal" in the applications menu
- Or right-click on the desktop and select "Open Terminal"

When you open the terminal, you will see a prompt that looks something like:

```
student@ubuntu:~$
```

This prompt shows:

- `student` - your username
- `ubuntu` - the hostname (computer name)
- `~` - your current directory (~ means home directory)
- `$` - indicates you are a regular user (# indicates root user)

Part 5: Basic Linux Commands

5.1 Navigation Commands

pwd (Print Working Directory)

Shows the absolute path of your current directory.

```
pwd
```

Example output: `/home/student`

ls (List)

Lists files and directories in the current directory.

```
ls
```

Common options:

- `ls -l` : Long format showing permissions, owner, size, and modification date
- `ls -a` : Show all files including hidden files (files starting with .)
- `ls -lh` : Long format with human-readable file sizes
- `ls -la` : Combine options to show all files in long format

Example:

```
ls -la
```

cd (Change Directory)

Changes your current directory.

```
cd /path/to/directory
```

Examples:

```
cd /home            # Go to /home directory
cd Documents        # Go to Documents in current directory
cd ..               # Go up one level to parent directory
cd ~                # Go to your home directory
cd -                # Go to previous directory
cd /                # Go to root directory
```

5.2 File and Directory Operations

mkdir (Make Directory)

Creates a new directory.

```
mkdir directory_name
```

Create nested directories:

```
mkdir -p parent/child/grandchild
```

The `-p` flag creates parent directories if they do not exist.

touch

Creates an empty file or updates the timestamp of an existing file.

```
touch filename.txt
```

Create multiple files:

```
touch file1.txt file2.txt file3.txt
```

cp (Copy)

Copies files or directories.

```
cp source destination
```

Examples:

```
cp file.txt backup.txt              # Copy file
cp file.txt /home/student/backup/   # Copy to another directory
cp -r folder/ backup_folder/        # Copy directory recursively
```

The `-r` flag is required when copying directories.

mv (Move)

Moves or renames files and directories.

```
mv source destination
```

Examples:

```
mv old_name.txt new_name.txt        # Rename file
mv file.txt /home/student/docs/     # Move file
mv folder/ /home/student/backup/    # Move directory
```

rm (Remove)

Deletes files or directories permanently. Be careful - there is no recycle bin!

```
rm filename
```

Examples:

```
rm file.txt                 # Delete a file
rm -r folder/               # Delete directory recursively
rm -f file.txt              # Force delete without confirmation
rm -rf folder/              # Force delete directory (use with extreme caution!)
```

Important: Never run `rm -rf /` or `rm -rf /*` as this will delete your entire system!

rmdir (Remove Directory)

Removes empty directories only.

```
rmdir empty_directory
```

5.3 Viewing File Contents

cat (Concatenate)

Displays entire file contents.

```
cat filename
```

Concatenate multiple files:

```
cat file1.txt file2.txt
```

less

Views file content one page at a time. Better for large files.

```
less filename
```

Navigation in less:

- Space: Next page
- b: Previous page
- /search_term: Search forward
- q: Quit

head

Shows the first 10 lines of a file.

```
head filename
```

Show first n lines:

```
head -n 20 filename
```

tail

Shows the last 10 lines of a file.

```
tail filename
```

Show last n lines:

```
tail -n 20 filename
```

Follow a file in real-time (useful for log files):

```
tail -f logfile.txt
```

wc (Word Count)

Counts lines, words, and characters in a file.

```
wc filename
```

Count only lines:

```
wc -l filename
```

5.4 Getting Help

man (Manual)

Displays the manual page for a command.

```
man command_name
```

Example:

```
man ls
```

Navigate using arrow keys, search with `/`, and quit with `q`.

--help

Most commands support a --help flag for quick reference.

```
ls --help
```

whatis

Gives a brief description of a command.

```
whatis ls
```

Part 6: Practical Exercises

Now that you have learned the basic commands, complete the following exercises to reinforce your understanding.

Exercise 1: Exploring the Filesystem

1. Open the terminal
2. Use `pwd` to see your current location
3. Use `ls` to see what files and directories are in your home directory
4. Navigate to the root directory using `cd /`
5. List the contents of the root directory using `ls`
6. Navigate to the /etc directory using `cd /etc`
7. List the contents using `ls -l`
8. Return to your home directory using `cd ~`
9. Verify you are home using `pwd`

Exercise 2: Creating a Directory Structure

1. In your home directory, create a directory called `devops_lab`
2. Navigate into the `devops_lab` directory
3. Inside `devops_lab`, create three directories: `week1`, `week2`, `week3`
4. List the contents to verify the directories were created
5. Navigate into `week1`
6. Create a subdirectory called `notes`
7. Use `pwd` to confirm your current location
8. Go back to the `devops_lab` directory using `cd ..`

Exercise 3: Working with Files

1. Navigate to `devops_lab/week1`
2. Create a file called `intro.txt` using touch
3. Create three more files: `commands.txt`, `filesystem.txt`, `practice.txt`
4. List all files to verify creation
5. Copy `intro.txt` to `intro_backup.txt`
6. Rename `practice.txt` to `exercises.txt` using mv
7. List the files again to see the changes
8. Delete `intro_backup.txt`
9. Verify the file was deleted by listing files again

Exercise 4: File Content Operations

1. Create a new file called `my_notes.txt`
2. Open it with a text editor (nano or vim) and write: "DevOps is a set of practices that combines software development and IT operations."
3. Save and close the file
4. Display the contents using `cat my_notes.txt`
5. Add more content to the file about what you learned today
6. Use `cat my_notes.txt` again to verify
7. Use `wc -l my_notes.txt` to count how many lines you have written
8. Use `head -n 3 my_notes.txt` to display the first 3 lines

Exercise 5: Advanced Navigation

1. From anywhere, navigate directly to `/usr/bin` using an absolute path
2. List the files (you will see many command programs here)
3. Navigate to `/var/log` using an absolute path
4. List the log files here
5. Navigate back to your home directory using `cd ~`
6. Create a directory structure: `projects/web/frontend` using a single mkdir command
7. Navigate directly into the frontend directory
8. Use `pwd` to confirm your location
9. Go back to your home directory

Part 7: Understanding Command Structure

7.1 Anatomy of a Linux Command

A typical Linux command follows this structure:

```
command [options] [arguments]
```

Command: The program you want to run (e.g., ls, cp, mkdir)

Options: Modify the behavior of the command. Usually start with - (single dash) for short options or -- (double dash) for long options.

Arguments: What the command operates on (e.g., filenames, directories)

Example:

```
ls -la /home/student
```

- `ls` is the command
- `-la` are options (combined -l and -a)
- `/home/student` is the argument

7.2 Wildcards and Pattern Matching

Wildcards help you work with multiple files at once.

*** (Asterisk):** Matches any number of characters

```
ls *.txt          # List all .txt files
rm file*          # Remove all files starting with "file"
```

? (Question Mark): Matches exactly one character

```
ls file?.txt      # Matches file1.txt, fileA.txt, but not file10.txt
```

[] (Brackets): Matches any one character inside the brackets

```
ls file[123].txt    # Matches file1.txt, file2.txt, file3.txt
ls file[a-z].txt    # Matches any file followed by one lowercase letter
```

7.3 Command Chaining

You can run multiple commands in sequence:

Semicolon (;): Run commands one after another, regardless of success

```
cd Documents; ls; pwd
```

AND (&&): Run the next command only if the previous one succeeds

```
mkdir test && cd test
```

OR (||): Run the next command only if the previous one fails

```
cd nonexistent || echo "Directory not found"
```

Part 8: Common Mistakes and Troubleshooting

8.1 Case Sensitivity

Linux is case-sensitive. `File.txt`, `file.txt`, and `FILE.TXT` are three different files.

```
ls File.txt          # Different from
ls file.txt
```

8.2 Spaces in Filenames

Spaces in filenames require special handling. Use quotes or escape the space with a backslash.

```
touch "my file.txt"     # Using quotes
touch my\ file.txt      # Using backslash
```

Better practice: Avoid spaces in filenames. Use underscores or hyphens instead.

```
touch my_file.txt       # Recommended
touch my-file.txt       # Also recommended
```

8.3 Hidden Files

Files starting with a dot (.) are hidden and will not show with `ls` unless you use the `-a` option.

```
ls                  # Won't show .hidden
ls -a               # Will show .hidden
```

8.4 Permission Denied Errors

If you see "Permission denied", you might be trying to access files or directories you do not have permission to read or modify. You will learn about permissions in the next lab.

8.5 Command Not Found

If you see "command not found", check for typos. Remember Linux is case-sensitive. The command might also not be installed on your system.

Lab Assessment Tasks

Complete the following tasks and submit screenshots or command outputs as directed by your instructor.

Task 1: Create the following directory structure in your home directory:

```
devops_course/
  ├── labs/
  │   ├── lab1/
  │   ├── lab2/
  │   └── lab3/
  └── assignments/
      ├── assignment1/
      └── assignment2/
```

Task 2: Navigate to `devops_course/labs/lab1` and create five files: `intro.txt`, `commands.txt`, `filesystem.txt`, `notes.txt`, and `summary.txt`

Task 3: Write a brief summary (3-4 lines) about what DevOps is in the `summary.txt` file using a text editor

Task 4: Copy all .txt files from lab1 to lab2

Task 5: Rename `summary.txt` in lab2 to `devops_summary.txt`

Task 6: Delete the `lab3` directory

Task 7: Create a file called `evidence.txt` in your home directory that contains the output of the following commands:

- `pwd`
- `ls -la devops_course`
- `cat devops_course/labs/lab1/summary.txt`

Lab 2: Linux Files, Permissions & Process Management

Lab Overview

This lab builds upon your foundational Linux knowledge by introducing file permissions, user and group management, and process monitoring. Understanding these concepts is critical for managing servers, controlling access to resources, and troubleshooting application issues in DevOps environments.

Topics Covered:

- File operations and manipulation
- File permissions and ownership
- User and group management
- Process monitoring and management
- System resource management

Learning Objectives

By the end of this lab, you will be able to:

1. Perform advanced file operations including searching and finding files
2. Understand and modify file permissions using chmod
3. Manage file ownership with chown and chgrp
4. Create and manage users and groups
5. Monitor running processes and system resources
6. Manage processes using kill, killall, and background jobs

Part 1: Advanced File Operations

1.1 Finding Files and Directories

find Command

The find command searches for files and directories in a directory hierarchy based on various criteria.

Basic syntax:

```
find [path] [options] [expression]
```

Examples:

```
find . -name "file.txt"          # Find file by exact name in current directory
```

```
find /home -name "*.txt"            # Find all .txt files in /home
find . -type f -name "*.log"        # Find only files (not directories)
find . -type d -name "temp"         # Find only directories
find . -size +10M                   # Find files larger than 10MB
find . -mtime -7                    # Find files modified in last 7 days
find . -user student                # Find files owned by user "student"
```

locate Command

Faster than find but uses a database that needs to be updated.

```
locate filename                     # Quick search for filename
sudo updatedb                       # Update the locate database
```

which Command

Shows the full path of shell commands.

```
which python                        # Shows where python is installed
which ls                            # Shows path to ls command
```

1.2 Searching Inside Files

grep (Global Regular Expression Print)

Searches for patterns within files.

Basic syntax:

```
grep [options] pattern [files]
```

Examples:

```
grep "error" logfile.txt            # Search for "error" in file
grep -i "error" logfile.txt         # Case-insensitive search
grep -r "function" .                # Recursive search in all files
grep -n "TODO" script.sh            # Show line numbers
grep -v "error" logfile.txt         # Show lines that DON'T contain "error"
grep -c "warning" logfile.txt       # Count matching lines
```

Combining grep with other commands:

```
ls -l | grep "txt"                  # List only .txt files
cat file.txt | grep "important"     # Search in output
```

1.3 File Comparison

diff Command

Compares two files line by line.

```
diff file1.txt file2.txt          # Show differences
diff -u file1.txt file2.txt       # Unified format (more readable)
diff -r dir1/ dir2/               # Compare directories recursively
```

1.4 File Compression and Archiving

tar (Tape Archive)

Creates and extracts archive files.

Creating archives:

```
tar -cvf archive.tar files/          # Create tar archive
tar -czvf archive.tar.gz files/      # Create compressed (gzip) archive
tar -cjvf archive.tar.bz2 files/     # Create compressed (bzip2) archive
```

Extracting archives:

```
tar -xvf archive.tar              # Extract tar archive
tar -xzvf archive.tar.gz          # Extract gzip archive
tar -xjvf archive.tar.bz2         # Extract bzip2 archive
```

Common options:

- c: create
- x: extract
- v: verbose (show progress)
- f: file (specify filename)
- z: gzip compression
- j: bzip2 compression

gzip and gunzip

Compress individual files:

```
gzip file.txt                     # Compresses to file.txt.gz
gunzip file.txt.gz                # Decompresses to file.txt
```

zip and unzip

```
zip archive.zip file1.txt file2.txt  # Create zip archive
zip -r archive.zip folder/           # Zip directory recursively
unzip archive.zip                    # Extract zip archive
unzip -l archive.zip                 # List contents without extracting
```

Part 2: Understanding File Permissions

2.1 Linux Permission Model

Every file and directory in Linux has three types of permissions for three categories of users.

Permission Types:

- **r (read)**: Permission to read the file content or list directory contents
- **w (write)**: Permission to modify the file or create/delete files in a directory
- **x (execute)**: Permission to execute a file or enter a directory

User Categories:

- **u (user/owner)**: The user who owns the file
- **g (group)**: Users who are members of the file's group
- **o (others)**: All other users
- **a (all)**: All three categories combined

2.2 Viewing Permissions

Use `ls -l` to view detailed file information including permissions:

```
ls -l file.txt
```

Output example:

```
-rw-r--r-- 1 student student 1234 Jan 20 10:30 file.txt
```

Let's break this down:

```
-rw-r--r--
│ │ │ │ │ │ │ │ └─ Others: read
│ │ │ │ │ │ │ └── Others: no write
│ │ │ │ │ │ └─── Others: no execute
│ │ │ │ │ └──── Group: read
│ │ │ │ └───── Group: no write
│ │ │ └────── Group: no execute
│ │ └─────── Owner: read
```

```
│ │ └──────────── Owner: write
│ └──────────────── Owner: no execute
└──────────────────── File type (- for file, d for directory, l for link)
```

The number following permissions (1) is the number of hard links. Then comes the owner (student), group (student), file size (1234 bytes), date, and filename.

2.3 Numeric (Octal) Representation

Permissions can also be represented numerically:

- r (read) = 4
- w (write) = 2
- x (execute) = 1

Add the values to get the permission for each category:

- 0 = no permissions (---)
- 1 = execute only (--x)
- 2 = write only (-w-)
- 3 = write and execute (-wx)
- 4 = read only (r--)
- 5 = read and execute (r-x)
- 6 = read and write (rw-)
- 7 = read, write, and execute (rwx)

Example: 755 means:

- Owner: 7 (rwx) - read, write, execute
- Group: 5 (r-x) - read, execute
- Others: 5 (r-x) - read, execute

Common permission sets:

- 644: rw-r--r-- (standard file)
- 755: rwxr-xr-x (executable file)
- 700: rwx------ (private file)
- 777: rwxrwxrwx (everyone can do everything - generally not recommended)

2.4 Changing Permissions with chmod

Symbolic Mode:

```
chmod u+x file.sh          # Add execute for owner
chmod g-w file.txt         # Remove write for group
chmod o+r file.txt         # Add read for others
```

```
chmod a+x script.sh              # Add execute for all
chmod u+rw,g+r,o-rwx file.txt      # Multiple changes at once
```

Numeric Mode:

```
chmod 644 file.txt               # rw-r--r--
chmod 755 script.sh              # rwxr-xr-x
chmod 700 private.txt            # rwx------
chmod 777 shared.txt             # rwxrwxrwx (be careful!)
```

Recursive Changes:

```
chmod -R 755 directory/          # Apply to directory and all contents
```

2.5 Changing Ownership

chown (Change Owner)

Changes the owner of a file or directory. Requires root privileges.

```
sudo chown newowner file.txt        # Change owner
sudo chown newowner:newgroup file.txt # Change owner and group
sudo chown -R newowner directory/    # Recursive change
```

chgrp (Change Group)

Changes the group ownership.

```
sudo chgrp newgroup file.txt        # Change group
sudo chgrp -R newgroup directory/    # Recursive change
```

2.6 Special Permissions

Setuid (Set User ID):

When set on an executable, it runs with the permissions of the file owner, not the user running it.

```
chmod u+s executable             # Symbolic
chmod 4755 executable            # Numeric (4 prefix)
```

Setgid (Set Group ID):

When set on a directory, files created within inherit the directory's group.

```
chmod g+s directory              # Symbolic
```

```
chmod 2755 directory          # Numeric (2 prefix)
```

Sticky Bit:

When set on a directory, only the file owner can delete their files (useful for /tmp).

```
chmod +t directory            # Symbolic
chmod 1755 directory          # Numeric (1 prefix)
```

Part 3: User and Group Management

3.1 Understanding Users and Groups

In Linux, every user has:

- A unique username
- A unique User ID (UID)
- A primary group
- Optionally, membership in multiple secondary groups
- A home directory
- A login shell

Users are defined in `/etc/passwd` Groups are defined in `/etc/group` Encrypted passwords are stored in `/etc/shadow` (root only)

3.2 User Information Commands

whoami

Shows your current username.

```
whoami
```

id

Shows user ID, group ID, and group memberships.

```
id                    # Your information
id username           # Another user's information
```

who

Shows who is currently logged in.

```
who
```

w

Shows who is logged in and what they are doing.

```
w
```

users

Lists currently logged-in users.

```
users
```

3.3 Creating and Managing Users

adduser (High-level command)

Interactive way to create users.

```
sudo adduser newuser              # Creates user with home directory
```

This command prompts for password and user information.

useradd (Low-level command)

More control but requires manual setup.

```
sudo useradd -m -s /bin/bash newuser # -m creates home, -s sets shell
sudo passwd newuser                  # Set password
```

Modifying Users:

```
sudo usermod -c "Full Name" username # Change full name
sudo usermod -d /new/home username   # Change home directory
sudo usermod -s /bin/zsh username    # Change shell
sudo usermod -l newname oldname      # Rename user
```

Deleting Users:

```
sudo userdel username              # Delete user (keeps home directory)
sudo userdel -r username           # Delete user and home directory
```

3.4 Creating and Managing Groups

groupadd

Creates a new group.

```
sudo groupadd developers          # Create group
sudo groupadd -g 1500 testers     # Create with specific GID
```

Adding Users to Groups:

```
sudo usermod -aG groupname username  # Add user to group (-a appends)
sudo gpasswd -a username groupname   # Alternative method
```

Removing Users from Groups:

```
sudo gpasswd -d username groupname   # Remove user from group
```

Viewing Group Members:

```
getent group groupname            # Show group members
groups username                   # Show user's groups
```

Deleting Groups:

```
sudo groupdel groupname           # Delete group
```

3.5 Switching Users

su (Switch User)

Switches to another user account.

```
su username                       # Switch user (keeps current environment)
su - username                     # Switch user (loads user's environment)
su -                              # Switch to root
exit                              # Return to previous user
```

sudo (Superuser Do)

Executes a single command with root privileges.

```
sudo command                      # Run command as root
sudo -u username command          # Run command as another user
sudo -i                           # Start root shell
```

Part 4: Process Management

4.1 Understanding Processes

A process is a running instance of a program. Every process has:

- A unique Process ID (PID)
- A Parent Process ID (PPID)
- Owner (user who started it)
- Priority level
- Resource usage (CPU, memory)
- State (running, sleeping, stopped, zombie)

4.2 Viewing Processes

ps (Process Status)

Shows currently running processes.

```
ps                  # Your processes
ps aux              # All processes, detailed
ps -ef              # All processes, full format
ps -u username      # Processes by user
ps -p PID           # Specific process
```

Common columns:

- PID: Process ID
- PPID: Parent Process ID
- USER: Process owner
- %CPU: CPU usage percentage
- %MEM: Memory usage percentage
- STAT: Process state
- COMMAND: Command that started process

top

Real-time, dynamic view of processes.

```
top
```

Useful keys while in top:

- q: Quit
- k: Kill a process (prompts for PID)
- M: Sort by memory usage
- P: Sort by CPU usage
- h: Help
- 1: Show individual CPU cores

htop

Enhanced version of top (may need installation).

```
sudo apt install htop          # Install on Ubuntu
htop                           # Run htop
```

pgrep

Search for processes by name.

```
pgrep firefox                  # Find PIDs of firefox processes
pgrep -u username              # Processes by user
pgrep -l firefox               # Show PID and name
```

4.3 Process States

- **R (Running)**: Currently executing
- **S (Sleeping)**: Waiting for an event
- **D (Uninterruptible Sleep)**: Usually waiting for I/O
- **T (Stopped)**: Suspended
- **Z (Zombie)**: Finished but not cleaned up by parent

4.4 Managing Processes

kill

Sends signals to processes to control them.

```
kill PID                       # Send TERM signal (graceful shutdown)
kill -9 PID                    # Send KILL signal (force kill)
kill -STOP PID                 # Pause process
kill -CONT PID                 # Resume process
```

Common signals:

- 1 (HUP): Hangup, reload configuration
- 9 (KILL): Force kill (cannot be ignored)
- 15 (TERM): Graceful termination (default)
- 19 (STOP): Pause process
- 18 (CONT): Continue process

killall

Kills processes by name.

```
killall firefox                # Kill all firefox processes
killall -9 unresponsive_app    # Force kill by name
```

pkill

Similar to killall but more flexible.

```
pkill firefox                  # Kill processes by name
pkill -u username              # Kill all processes by user
```

4.5 Background and Foreground Jobs

Running Commands in Background:

```
command &                      # Run in background
```

Job Control:

```
jobs                           # List background jobs
fg %1                          # Bring job 1 to foreground
bg %1                          # Resume job 1 in background
Ctrl+Z                         # Suspend current foreground job
```

nohup (No Hangup)

Runs command immune to hangups (continues after logout).

```
nohup command &                # Run in background, ignore hangup
nohup python script.py > output.log & # Redirect output to file
```

disown

Removes job from shell's job table (makes it independent).

```
command &
disown                         # Current job won't receive hangup signal
```

4.6 Process Priority

nice

Starts a process with modified scheduling priority (-20 highest, 19 lowest).

```
nice -n 10 command             # Start with lower priority
nice -n -10 command            # Start with higher priority (needs sudo)
```

renice

Changes priority of running process.

```
renice -n 5 -p PID          # Change priority of PID
sudo renice -n -5 -p PID    # Increase priority (needs sudo)
```

Part 5: System Resource Management

5.1 Disk Usage

df (Disk Free)

Shows filesystem disk space usage.

```
df              # Show disk usage
df -h           # Human-readable format
df -T           # Show filesystem type
df /home        # Show specific filesystem
```

du (Disk Usage)

Shows directory space usage.

```
du              # Current directory
du -h           # Human-readable
du -sh          # Summary only
du -sh *        # Size of each item
du -h --max-depth=1    # Limit subdirectory depth
```

5.2 Memory Usage

free

Shows memory usage.

```
free            # Show memory in KB
free -h         # Human-readable
free -m         # Show in MB
free -g         # Show in GB
```

Output shows:

- total: Total installed memory
- used: Used memory
- free: Completely unused memory
- shared: Memory used by tmpfs
- buff/cache: Memory used for buffers and cache
- available: Estimation of memory available for new applications

5.3 System Information

uname

Shows system information.

```
uname -a                    # All information
uname -r                    # Kernel version
uname -m                    # Machine hardware name
```

uptime

Shows how long system has been running.

```
uptime
```

lscpu

Shows CPU architecture information.

```
lscpu
```

lsblk

Lists block devices (disks and partitions).

```
lsblk
lsblk -f                    # Show filesystem type
```

dmesg

Shows kernel ring buffer messages (useful for hardware issues).

```
dmesg                       # Show all messages
dmesg | tail                # Show recent messages
dmesg | grep -i error       # Show errors
```

5.4 Monitoring System Logs

journalctl (systemd systems)

Views systemd logs.

```
journalctl                    # All logs
journalctl -f                 # Follow logs (like tail -f)
journalctl -u servicename     # Logs for specific service
journalctl --since today      # Today's logs
journalctl --since "2026-01-20"  # Logs from specific date
journalctl -p err             # Error level logs only
```

Traditional log files:

```
tail -f /var/log/syslog       # System log
tail -f /var/log/auth.log     # Authentication log
tail -f /var/log/kern.log     # Kernel log
```

Part 6: Practical Exercises

Exercise 1: File Permissions Practice

1. Create a directory called `permissions_lab` in your home directory
2. Navigate into it and create three files: `public.txt`, `private.txt`, `executable.sh`
3. View the default permissions using `ls -l`
4. Set permissions on `public.txt` to 644 (rw-r--r--)
5. Set permissions on `private.txt` to 600 (rw-------)
6. Set permissions on `executable.sh` to 755 (rwxr-xr-x)
7. Verify the changes with `ls -l`
8. Try to create a file that only you can read and write: `secret.txt` with permissions 600
9. Create a shell script that prints "Hello DevOps" and make it executable
10. Run your script: `./executable.sh`

Exercise 2: User and Group Management

Note: These commands require sudo privileges. Your instructor may provide a virtual machine or special account.

1. Create a new group called `devteam`
2. Create a new user called `developer1`
3. Add `developer1` to the `devteam` group
4. Create a directory called `team_project` with group ownership set to `devteam`
5. Set permissions on `team_project` so group members can read, write, and execute
6. Verify the group membership with `groups developer1`
7. Use `id developer1` to see all user information

Exercise 3: Process Management

1. Open the terminal and run `top` to see running processes
2. Press 'q' to quit top
3. Start a long-running process in the background: `sleep 300 &`
4. Use `jobs` to see your background jobs
5. Use `ps aux | grep sleep` to find the PID of your sleep command
6. Kill the sleep process using `kill PID` (replace PID with actual number)
7. Verify it's gone with `ps aux | grep sleep`
8. Start another sleep process: `sleep 200 &`
9. Use `killall sleep` to kill it
10. Try running: `htop` to see an enhanced process viewer (if available)

Exercise 4: Finding and Searching Files

1. Create a directory structure:

```
search_lab/
├── documents/
│   ├── report.txt
│   └── notes.txt
├── scripts/
│   ├── backup.sh
│   └── deploy.sh
└── logs/
    ├── error.log
    └── access.log
```

2. Add the word "ERROR" to error.log
3. Add the word "SUCCESS" to access.log
4. Use `find` to locate all .log files: `find search_lab -name "*.log"`
5. Use `find` to locate all .sh files
6. Use `grep` to find which log file contains "ERROR": `grep -r "ERROR" search_lab/logs/`
7. Use `grep` to count how many times "ERROR" appears: `grep -c "ERROR" search_lab/logs/error.log`

Exercise 5: System Monitoring

1. Check disk usage: `df -h`
2. Check memory usage: `free -h`
3. Check your home directory size: `du -sh ~`
4. Find the top 5 largest directories in your home: `du -h ~ | sort -rh | head -5`
5. Check system uptime: `uptime`
6. View CPU information: `lscpu`

7. List all block devices: `lsblk`
8. Check which processes are using the most CPU: `ps aux --sort=-%cpu | head -10`

Exercise 6: File Compression

1. Create a directory called `backup_test` with several files inside
2. Create a tar archive: `tar -cvf backup.tar backup_test/`
3. Create a compressed tar archive: `tar -czvf backup.tar.gz backup_test/`
4. Compare the sizes: `ls -lh backup.tar backup.tar.gz`
5. Delete the `backup_test` directory
6. Extract the tar.gz archive: `tar -xzvf backup.tar.gz`
7. Verify the files were restored
8. Create a zip archive: `zip -r backup.zip backup_test/`
9. List the contents without extracting: `unzip -l backup.zip`

Lab Assessment Tasks

Complete the following tasks and document your work:

Task 1: Create a shell script named `system_info.sh` that displays:

- Current user
- Current directory
- Disk usage of home directory
- Memory usage
- Top 5 processes by CPU usage

Make the script executable and run it.

Task 2: Create a directory structure for a web project:

```
webapp/
├── public/
├── src/
├── config/
└── logs/
```

Set the following permissions:

- `public/`: 755
- `src/`: 750
- `config/`: 700
- `logs/`: 775

Task 3: Create three files:

- `readme.txt` with permissions 644
- `deploy.sh` with permissions 755
- `secrets.conf` with permissions 600

Write a command that lists only files with executable permissions.

Task 4: Start a long-running process (e.g., `sleep 500`) in the background. Find its PID using three different methods:

- Using `ps` with grep
- Using `pgrep`
- Using `jobs` and `ps`

Then kill the process.

Task 5: Find all `.txt` files in your home directory that were modified in the last 7 days. Save the output to a file called `recent_files.txt`.
Task 6: Create a compressed backup of your `devops_course` directory (from Lab 1) named `devops_backup_YYYYMMDD.tar.gz` (use today's date).

Lab 3: Shell Scripting, Git Basics & Application Concepts

Lab Overview

This lab introduces you to Bash shell scripting for task automation, Git version control for code management, and fundamental application architecture concepts. You will learn to write scripts that automate repetitive tasks, use Git for tracking code changes locally, and understand how modern applications are structured with frontend and backend components.

Topics Covered:

- Bash scripting fundamentals
- Variables, conditionals, and loops in shell scripts
- Git installation and configuration
- Local Git workflows (init, add, commit, log)
- Introduction to frontend/backend architecture
- Understanding client-server model

Learning Objectives

By the end of this lab, you will be able to:

1. Write Bash scripts to automate common system administration tasks
2. Use variables, conditionals, and loops in shell scripts
3. Initialize Git repositories and manage version history
4. Perform basic Git operations (add, commit, status, log)
5. Explain the difference between frontend and backend components
6. Understand how applications communicate in a client-server model

Part 1: Introduction to Shell Scripting

1.1 What is Shell Scripting?

A shell script is a text file containing a sequence of commands that the shell can execute. Shell scripts allow you to automate repetitive tasks, combine multiple commands, and create custom tools for system administration.

Benefits of Shell Scripting:

- Automates repetitive tasks
- Reduces human error
- Saves time on routine operations
- Creates reusable utilities
- Improves system administration efficiency

1.2 Creating Your First Script

Shell scripts typically start with a shebang line that tells the system which interpreter to use.

Basic Script Structure:

bash

```bash
#!/bin/bash
# This is a comment
echo "Hello, DevOps!"
```

Creating and Running a Script:

bash

```bash
# Create a new script file
nano hello.sh

# Make it executable
chmod +x hello.sh

# Run the script
./hello.sh
```

The `#!/bin/bash` line is called a shebang. It tells the system to use the Bash interpreter to execute the script.

1.3 Variables in Bash

Variables store data that can be used throughout your script.

Defining Variables:

bash

```bash
#!/bin/bash

# Variable assignment (no spaces around =)
name="DevOps Student"
age=20
course="SE202"
```

```
# Using variables (use $ to access value)
echo "Name: $name"
echo "Age: $age"
echo "Course: $course"
```

Important Rules:

- No spaces around the = sign
- Variable names are case-sensitive
- Use $ to access variable values
- Quote strings with spaces

Reading User Input:

bash

```
#!/bin/bash

echo "Enter your name:"
read username
echo "Hello, $username!"
```

Command Substitution:

Store command output in variables:

bash

```
#!/bin/bash

current_date=$(date)
current_user=$(whoami)
current_dir=$(pwd)

echo "Date: $current_date"
echo "User: $current_user"
echo "Directory: $current_dir"
```

1.4 Conditional Statements

Conditionals allow scripts to make decisions.

If Statement Syntax:

bash

```bash
#!/bin/bash

if [ condition ]; then
   # commands
fi
```

If-Else Example:

bash

```bash
#!/bin/bash

echo "Enter a number:"
read number

if [ $number -gt 10 ]; then
   echo "Number is greater than 10"
else
   echo "Number is 10 or less"
fi
```

Comparison Operators:

For numbers:

- `-eq` : equal to
- `-ne` : not equal to
- `-gt` : greater than
- `-lt` : less than
- `-ge` : greater than or equal to
- `-le` : less than or equal to

For strings:

- `=` : equal to
- `!=` : not equal to
- `-z` : string is empty
- `-n` : string is not empty

Multiple Conditions:

bash

```bash
#!/bin/bash

echo "Enter your age:"
read age

if [ $age -lt 18 ]; then
   echo "You are a minor"
elif [ $age -ge 18 ] && [ $age -lt 65 ]; then
   echo "You are an adult"
else
   echo "You are a senior citizen"
fi
```

File Test Operators:

bash

```bash
#!/bin/bash

file="test.txt"

if [ -f $file ]; then
   echo "File exists"
else
   echo "File does not exist"
fi
```

Common file tests:

- `-f` : file exists and is a regular file
- `-d` : directory exists
- `-r` : file is readable
- `-w` : file is writable
- `-x` : file is executable

1.5 Loops in Bash

Loops allow you to repeat commands multiple times.

For Loop:

bash

```bash
#!/bin/bash

# Loop through a list
for name in Alice Bob Charlie
do
    echo "Hello, $name"
done

# Loop through numbers
for i in {1..5}
do
    echo "Number: $i"
done

# Loop through files
for file in *.txt
do
    echo "Found file: $file"
done
```

While Loop:

bash

```bash
#!/bin/bash

counter=1

while [ $counter -le 5 ]
do
    echo "Counter: $counter"
    counter=$((counter + 1))
done
```

Reading Files Line by Line:

bash

```bash
#!/bin/bash
```

```
while read line
do
    echo "Line: $line"
done < input.txt
```

1.6 Functions in Bash

Functions help organize code into reusable blocks.

bash

```bash
#!/bin/bash

# Define a function
greet() {
    echo "Hello, $1!"
}

# Call the function
greet "DevOps Student"
greet "World"

# Function with return value
add_numbers() {
    local result=$(($1 + $2))
    echo $result
}

sum=$(add_numbers 5 3)
echo "Sum: $sum"
```

1.7 Script Arguments

Scripts can accept command-line arguments.

bash

```bash
#!/bin/bash

echo "Script name: $0"
echo "First argument: $1"
```

```
echo "Second argument: $2"
echo "All arguments: $@"
echo "Number of arguments: $#"
```

Run with: `./script.sh arg1 arg2`

Part 2: Git Basics

2.1 What is Git?

Git is a distributed version control system that tracks changes in source code during software development. It allows multiple developers to work on the same project, maintains a complete history of changes, and helps manage different versions of code.

Why Use Git?

- Track changes over time
- Collaborate with team members
- Revert to previous versions
- Create branches for features
- Industry standard tool

2.2 Installing and Configuring Git

Installation:

bash

```bash
# Ubuntu/Debian
sudo apt update
sudo apt install git

# Check installation
git --version
```

Initial Configuration:

bash

```bash
# Set your name
git config --global user.name "Your Name"

# Set your email
git config --global user.email "your.email@example.com"
```

```bash
# Set default branch name
git config --global init.defaultBranch main

# View configuration
git config --list
```

2.3 Creating a Git Repository

Method 1: Initialize a New Repository

```bash
# Create a project directory
mkdir my-first-repo
cd my-first-repo

# Initialize Git repository
git init

# Check status
git status
```

Understanding the .git Directory:

When you run `git init`, Git creates a hidden `.git` directory that stores all version control information. Never delete this directory manually.

2.4 Basic Git Workflow

The basic Git workflow consists of three main areas:

1. **Working Directory**: Where you modify files
2. **Staging Area (Index)**: Where you prepare changes for commit
3. **Repository**: Where Git permanently stores committed changes

The Workflow:

```bash
# 1. Create or modify files
echo "# My Project" > README.md
```

```bash
# 2. Check status
git status

# 3. Add files to staging area
git add README.md

# Or add all changes
git add .

# 4. Check status again
git status

# 5. Commit changes
git commit -m "Initial commit with README"

# 6. View commit history
git log
```

2.5 Git Status and Tracking

Understanding File States:

Files in a Git repository can be in one of four states:

1. **Untracked**: Git doesn't know about the file
2. **Unmodified**: File is tracked but hasn't changed
3. **Modified**: File has been changed but not staged
4. **Staged**: File is ready to be committed

bash

```bash
# Check detailed status
git status

# Short status format
git status -s
```

2.6 Staging Changes

The staging area allows you to selectively choose which changes to commit.

bash

```bash
# Stage a specific file
git add filename.txt

# Stage multiple files
git add file1.txt file2.txt

# Stage all changes in current directory
git add .

# Stage all txt files
git add *.txt

# Remove file from staging area (unstage)
git reset filename.txt
```

2.7 Making Commits

Commits are snapshots of your project at a specific point in time.

Good Commit Messages:

- Present tense: "Add feature" not "Added feature"
- Concise but descriptive
- Explain what and why, not how

bash

```bash
# Commit staged changes
git commit -m "Add user authentication feature"

# Commit with detailed message
git commit -m "Add login functionality" -m "Users can now log in with email and password"

# Stage and commit in one step (tracked files only)
git commit -am "Update documentation"
```

2.8 Viewing History

bash

```bash
# View commit history
git log
```

```bash
# Compact log (one line per commit)
git log --oneline

# Show last n commits
git log -n 5

# Show commits with changes
git log -p

# Show commits by author
git log --author="Your Name"

# Graphical representation
git log --graph --oneline
```

2.9 Viewing Changes

bash

```bash
# Show unstaged changes
git diff

# Show staged changes
git diff --staged

# Show changes in a specific file
git diff filename.txt

# Compare two commits
git diff commit1 commit2
```

2.10 Ignoring Files

Create a `.gitignore` file to tell Git which files to ignore:

bash

```bash
# Create .gitignore
nano .gitignore
```

```
# Example .gitignore contents:
# Ignore log files
*.log

# Ignore temporary files
*.tmp
*~

# Ignore node_modules directory
node_modules/

# Ignore environment files
.env

# But don't ignore .gitignore itself
!.gitignore
```

Then add and commit it:

```bash
git add .gitignore
git commit -m "Add gitignore file"
```

Part 3: Application Concepts

3.1 Understanding Applications

An application (or app) is a software program designed to perform specific tasks for users. In DevOps, we typically work with web applications, which are accessed through web browsers.

Types of Applications:

- **Desktop Applications**: Run locally on a computer (e.g., Microsoft Word)
- **Web Applications**: Accessed through browsers (e.g., Gmail, Facebook)
- **Mobile Applications**: Run on smartphones and tablets
- **Command-Line Applications**: Text-based interface (like the tools we've been using)

3.2 Client-Server Architecture

Most modern web applications follow a client-server model:

Client (Frontend):

- Runs in the user's web browser
- Handles user interface and user interactions
- Displays information to users
- Technologies: HTML, CSS, JavaScript

Server (Backend):

- Runs on a remote computer (server)
- Processes requests from clients
- Manages data and business logic
- Stores and retrieves data from databases
- Technologies: Python, Node.js, Java, Go

How They Communicate:

1. User interacts with frontend in browser
2. Frontend sends HTTP request to backend server
3. Backend processes the request
4. Backend queries database if needed
5. Backend sends HTTP response back to frontend
6. Frontend displays the result to user

3.3 Frontend vs Backend

Frontend (Client-Side):

- What users see and interact with
- Presentation and user experience
- Runs in the user's browser
- Examples: buttons, forms, menus, animations

Backend (Server-Side):

- What users don't see
- Business logic and data processing
- Runs on the server
- Examples: authentication, database operations, calculations

Example Scenario - Login Process:

Frontend (Browser):

1. User enters username and password

2. User clicks "Login" button

3. JavaScript sends credentials to server

4. Displays "Logged in successfully" or error message

Backend (Server):

1. Receives username and password
2. Checks credentials against database
3. Creates session if credentials are valid
4. Sends success or failure response

3.4 HTTP Basics

HTTP (Hypertext Transfer Protocol) is the protocol used for communication between frontend and backend.

HTTP Request Methods:

- **GET**: Retrieve data (e.g., load a webpage)
- **POST**: Send data to server (e.g., submit a form)
- **PUT**: Update existing data
- **DELETE**: Delete data

HTTP Status Codes:

- **200 OK**: Request succeeded
- **201 Created**: Resource created successfully
- **400 Bad Request**: Invalid request
- **401 Unauthorized**: Authentication required
- **404 Not Found**: Resource doesn't exist
- **500 Internal Server Error**: Server error

3.5 Introduction to APIs

API (Application Programming Interface) is a way for different software components to communicate.

REST API Example:

```
GET /api/users          - Get all users
GET /api/users/123      - Get user with ID 123
POST /api/users         - Create new user
PUT /api/users/123      - Update user 123
DELETE /api/users/123   - Delete user 123
```

Part 4: Practical Exercises

Exercise 1: Basic Shell Scripts

Task 1: Create a greeting script

Create `greet.sh`:

bash

```bash
#!/bin/bash

echo "Enter your name:"
read name
echo "Welcome to DevOps, $name!"
echo "Today's date is: $(date +%Y-%m-%d)"
```

Make it executable and run:

bash

```bash
chmod +x greet.sh
./greet.sh
```

Task 2: System information script

Create `sysinfo.sh`:

bash

```bash
#!/bin/bash

echo "=== System Information ==="
echo "Hostname: $(hostname)"
echo "Current User: $(whoami)"
echo "Current Directory: $(pwd)"
echo "Disk Usage:"
df -h | grep "^/dev"
echo ""
echo "Memory Usage:"
free -h
```

Task 3: File backup script

Create `backup.sh`:

bash

```bash
#!/bin/bash

# Check if argument is provided
if [ $# -eq 0 ]; then
    echo "Usage: $0 <directory_to_backup>"
    exit 1
fi

source_dir=$1
backup_name="backup_$(date +%Y%m%d_%H%M%S).tar.gz"

echo "Creating backup of $source_dir..."
tar -czf $backup_name $source_dir

if [ $? -eq 0 ]; then
    echo "Backup created successfully: $backup_name"
else
    echo "Backup failed!"
    exit 1
fi
```

Exercise 2: Conditional Logic

Task 1: File checker script

Create `filecheck.sh`:

bash

```bash
#!/bin/bash

echo "Enter filename:"
read filename

if [ -f "$filename" ]; then
    echo "File exists!"
    echo "Size: $(du -h "$filename" | cut -f1)"
    echo "Permissions: $(ls -l "$filename" | cut -d' ' -f1)"
```

```
elif [ -d "$filename" ]; then
    echo "This is a directory, not a file"
else
    echo "File does not exist"
fi
```

Task 2: Grade calculator

Create `grade.sh`:

bash

```
#!/bin/bash

echo "Enter your score (0-100):"
read score

if [ $score -ge 85 ]; then
    echo "Grade: A"
elif [ $score -ge 70 ]; then
    echo "Grade: B"
elif [ $score -ge 60 ]; then
    echo "Grade: C"
elif [ $score -ge 50 ]; then
    echo "Grade: D"
else
    echo "Grade: F"
fi
```

Exercise 3: Loops

Task 1: File processor

Create `process_files.sh`:

bash

```
#!/bin/bash

echo "Creating test files..."
for i in {1..5}
```

```
do
    echo "This is file number $i" > file$i.txt
    echo "Created file$i.txt"
done

echo ""
echo "Listing created files:"
ls -l file*.txt
```

Task 2: Directory size report

Create `dirsize.sh`:

bash

```
#!/bin/bash

echo "=== Directory Size Report ==="
for dir in */
do
    size=$(du -sh "$dir" 2>/dev/null | cut -f1)
    echo "$dir: $size"
done
```

Exercise 4: Git Practice

Task 1: Create a project with Git

bash

```
# Create project directory
mkdir git-practice
cd git-practice

# Initialize Git
git init

# Create README file
echo "# Git Practice Project" > README.md
echo "This project is for learning Git basics." >> README.md
```

```bash
# Check status
git status

# Add and commit
git add README.md
git commit -m "Initial commit: Add README"

# View history
git log
```

Task 2: Make multiple commits

```bash
# Create a new file
echo "## Features" >> README.md
echo "- Feature 1" >> README.md

# Check what changed
git diff

# Commit the change
git add README.md
git commit -m "Add features section to README"

# Create a new file
echo "print('Hello, DevOps!')" > hello.py

# Add and commit
git add hello.py
git commit -m "Add Python hello script"

# View history
git log --oneline
```

Task 3: Practice ignoring files

```bash
# Create some files
```

```bash
touch debug.log
touch test.tmp
mkdir temp_data

# Create .gitignore
cat > .gitignore << EOF
*.log
*.tmp
temp_data/
EOF

# Check status (ignored files won't appear)
git status

# Commit .gitignore
git add .gitignore
git commit -m "Add gitignore file"
```

Exercise 5: Combining Scripts and Git

Task: Create a monitored script project

bash

```bash
# Create project
mkdir system-monitor
cd system-monitor
git init

# Create monitoring script
cat > monitor.sh << 'EOF'
#!/bin/bash

echo "=== System Monitor ==="
echo "Date: $(date)"
echo "Uptime: $(uptime -p)"
echo "Disk Usage:"
df -h | grep "^/dev" | awk '{print $1 ": " $5}'
echo "Top 5 Processes by CPU:"
```

```
ps aux --sort=-%cpu | head -6 | tail -5
EOF

chmod +x monitor.sh

# Commit initial version
git add monitor.sh
git commit -m "Add initial system monitor script"

# Enhance the script
cat >> monitor.sh << 'EOF'

echo ""
echo "Memory Usage:"
free -h | grep "Mem:" | awk '{print "Used: " $3 " / Total: " $2}'
EOF

# Commit enhancement
git add monitor.sh
git commit -m "Add memory usage monitoring"

# View history
git log --oneline
```

Part 5: Practical Application Scenarios

Scenario 1: Automated Log Cleanup

Create `cleanup_logs.sh`:

bash

```
#!/bin/bash

log_dir="/var/log"
days_old=7

echo "=== Log Cleanup Script ==="
echo "Searching for logs older than $days_old days in $log_dir"
```

```
# Find and list old log files
old_logs=$(find $log_dir -name "*.log" -type f -mtime +$days_old 2>/dev/null)

if [ -z "$old_logs" ]; then
    echo "No old log files found"
else
    echo "Found old log files:"
    echo "$old_logs"

    echo ""
    echo "Do you want to delete these files? (yes/no)"
    read answer

    if [ "$answer" = "yes" ]; then
        find $log_dir -name "*.log" -type f -mtime +$days_old -delete 2>/dev/null
        echo "Old logs deleted"
    else
        echo "Deletion cancelled"
    fi
fi
```

Scenario 2: User Management Script

Create `user_manager.sh`:

bash

```
#!/bin/bash

show_menu() {
    echo "=== User Management ==="
    echo "1. List all users"
    echo "2. Check if user exists"
    echo "3. Show current user"
    echo "4. Exit"
    echo "Enter choice:"
}
```

```
list_users() {
    echo "System users:"
    cut -d: -f1 /etc/passwd | sort
}

check_user() {
    echo "Enter username to check:"
    read username
    if id "$username" &>/dev/null; then
        echo "User $username exists"
        id "$username"
    else
        echo "User $username does not exist"
    fi
}

while true
do
    show_menu
    read choice

    case $choice in
        1) list_users ;;
        2) check_user ;;
        3) echo "Current user: $(whoami)" ;;
        4) echo "Goodbye!"; exit 0 ;;
        *) echo "Invalid choice" ;;
    esac

    echo ""
done
```

Lab Assessment Tasks

Complete the following tasks and document your work:

Task 1: Personal Information Script

Create a script called `myinfo.sh` that:

1. Asks for your name, student ID, and favorite programming language
2. Displays a formatted summary
3. Saves the information to a file called `student_info.txt`
4. Displays the current date and time
5. Shows your current directory

Task 2: File Organizer Script

Create `organize.sh` that:

1. Creates directories: `documents`, `images`, `scripts`, `others`
2. Moves `.txt` and `.pdf` files to `documents/`
3. Moves `.jpg` and `.png` files to `images/`
4. Moves `.sh` files to `scripts/`
5. Moves everything else to `others/`
6. Displays a summary of moved files

Task 3: Git Project Setup

Create a new project called `devops-scripts`:

1. Initialize a Git repository
2. Create a README.md describing the project
3. Create at least 3 shell scripts from previous exercises
4. Create a .gitignore file to ignore `.log` and `.tmp` files
5. Make at least 5 meaningful commits
6. Use `git log --oneline --graph` to display history
7. Save the output to `git-history.txt`

Task 4: System Health Check Script

Create `health_check.sh` that:

1. Checks if disk usage is above 80% (warn if true)
2. Checks if memory usage is above 80% (warn if true)
3. Lists top 5 CPU-consuming processes
4. Checks if system uptime is less than 1 day (warn if true)
5. Saves the report to `health_report_$(date +%Y%m%d).txt`
6. Uses conditional statements and functions

Task 5: Git Workflow Documentation

Create a document called `git-workflow.md` that:

1. Explains the purpose of Git
2. Lists and explains 10 Git commands you learned
3. Describes the three Git areas (working, staging, repository)
4. Provides examples of good commit messages
5. Explains when to use .gitignore

Submit all scripts with proper permissions and ensure they run without errors.

Lab 4: Frontend & Backend Fundamentals

Lab Overview

This lab provides hands-on experience with building basic frontend and backend applications. You will create simple web interfaces using HTML, CSS, and JavaScript, and develop backend services using Python Flask or Node.js Express. By the end of this lab, you will understand how client-server communication works and be able to build a complete web application.

Topics Covered:

- HTML and CSS basics for frontend
- JavaScript for interactivity
- Backend development with Python Flask or Node.js
- HTTP request/response cycle
- Running applications locally
- Basic API development

Learning Objectives

By the end of this lab, you will be able to:

1. Create basic web pages using HTML and CSS
2. Add interactivity with JavaScript
3. Build a simple backend API using Python Flask or Node.js Express
4. Understand HTTP methods (GET, POST) and status codes
5. Connect frontend and backend applications
6. Run and test web applications locally

Part 1: Frontend Basics - HTML

1.1 Introduction to HTML

HTML (HyperText Markup Language) is the standard markup language for creating web pages. It describes the structure and content of a webpage using elements and tags.

Basic HTML Structure:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Webpage</title>
```

```
</head>
<body>
    <h1>Hello, DevOps!</h1>
    <p>This is my first webpage.</p>
</body>
</html>
```

Create this file as `index.html` and open it in a web browser.

1.2 Common HTML Elements

Headings:

html

```
<h1>Main Heading</h1>
<h2>Subheading</h2>
<h3>Smaller Heading</h3>
<!-- h1 to h6 available -->
```

Paragraphs and Text Formatting:

html

```
<p>This is a paragraph.</p>
<p>This is another paragraph with <strong>bold text</strong> and <em>italic text</em>.</p>
<br> <!-- Line break -->
<hr> <!-- Horizontal rule -->
```

Lists:

html

```
<!-- Unordered list -->
<ul>
    <li>Item 1</li>
    <li>Item 2</li>
    <li>Item 3</li>
</ul>

<!-- Ordered list -->
<ol>
    <li>First item</li>
```

```
    <li>Second item</li>
    <li>Third item</li>
</ol>
```

Links:

html

```
<a href="https://www.example.com">Click here</a>
<a href="about.html">About Page</a>
<a href="#section1">Jump to Section 1</a>
```

Images:

html

```
<img src="image.jpg" alt="Description of image">
<img src="https://example.com/logo.png" alt="Logo" width="200">
```

Divs and Spans:

html

```
<div>
    <p>Divs are block-level containers</p>
</div>

<p>This is <span style="color: red;">inline text</span> in a paragraph</p>
```

1.3 HTML Forms

Forms are used to collect user input:

html

```
<form action="/submit" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>
    <br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>
    <br><br>
```

```
    <label for="message">Message:</label>
    <textarea id="message" name="message" rows="4"></textarea>
    <br><br>

    <button type="submit">Submit</button>
</form>
```

Common Input Types:

- `text`: Single-line text
- `email`: Email address
- `password`: Hidden password
- `number`: Numeric input
- `checkbox`: Checkbox
- `radio`: Radio button
- `date`: Date picker
- `file`: File upload

Part 2: Frontend Styling - CSS

2.1 Introduction to CSS

CSS (Cascading Style Sheets) is used to style and layout HTML elements.

Three Ways to Add CSS:

1. Inline CSS:

html

```
<p style="color: blue; font-size: 16px;">Styled paragraph</p>
```

2. Internal CSS:

html

```
<head>
    <style>
        p {
            color: blue;
            font-size: 16px;
        }
    </style>
</head>
```

3. External CSS (Recommended):

Create `style.css`:

```css
p {
    color: blue;
    font-size: 16px;
}
```

Link in HTML:

```html
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

2.2 CSS Selectors

```css
/* Element selector */
p {
    color: black;
}

/* Class selector */
.highlight {
    background-color: yellow;
}

/* ID selector */
#header {
    font-size: 24px;
}

/* Multiple selectors */
h1, h2, h3 {
    color: navy;
}
```

```
/* Descendant selector */
div p {
    margin-left: 20px;
}
```

2.3 Common CSS Properties

```
CSS
/* Text Styling */
.text-style {
    color: #333;
    font-size: 16px;
    font-family: Arial, sans-serif;
    font-weight: bold;
    text-align: center;
    text-decoration: underline;
}

/* Box Model */
.box {
    width: 300px;
    height: 200px;
    padding: 20px;
    margin: 10px;
    border: 2px solid black;
    background-color: lightblue;
}

/* Display and Layout */
.container {
    display: flex;
    justify-content: center;
    align-items: center;
}

/* Positioning */
```

```
.positioned {
    position: relative;
    top: 10px;
    left: 20px;
}
```

2.4 Creating a Styled Webpage

Create `styled-page.html`:

html

```
<!DOCTYPE html>
<html>
<head>
    <title>DevOps Portfolio</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }

        .container {
            max-width: 800px;
            margin: 50px auto;
            background-color: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }

        h1 {
            color: #2c3e50;
            border-bottom: 3px solid #3498db;
            padding-bottom: 10px;
        }
```

```
        .skill {
            display: inline-block;
            background-color: #3498db;
            color: white;
            padding: 5px 15px;
            margin: 5px;
            border-radius: 5px;
        }

        button {
            background-color: #2ecc71;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #27ae60;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>DevOps Student Portfolio</h1>
        <h2>About Me</h2>
        <p>I am learning DevOps practices and tools.</p>

        <h2>Skills</h2>
        <span class="skill">Linux</span>
        <span class="skill">Git</span>
        <span class="skill">Shell Scripting</span>
        <span class="skill">Docker</span>

        <h2>Contact</h2>
        <button onclick="alert('Contact feature coming soon!')">Contact Me</button>
```

```
    </div>
</body>
</html>
```

Part 3: Frontend Interactivity - JavaScript

3.1 Introduction to JavaScript

JavaScript adds interactivity to web pages. It runs in the browser and can manipulate HTML and CSS dynamically.

Adding JavaScript to HTML:

1. Inline JavaScript:

html

```html
<button onclick="alert('Hello!')">Click Me</button>
```

2. Internal JavaScript:

html

```html
<script>
    console.log('Page loaded');
    alert('Welcome to my site!');
</script>
```

3. External JavaScript:

Create `script.js`:

javascript

```javascript
console.log('External script loaded');
```

Link in HTML:

html

```html
<script src="script.js"></script>
```

3.2 JavaScript Basics

Variables:

javascript

```javascript
// Variable declarations
let name = "DevOps Student";
const age = 20;
var course = "SE202";  // Older way, use let instead

console.log(name);
console.log(age);
```

Functions:

javascript

```javascript
// Function declaration
function greet(name) {
    return "Hello, " + name + "!";
}

// Function call
let message = greet("Ali");
console.log(message);

// Arrow function (modern way)
const add = (a, b) => a + b;
console.log(add(5, 3));
```

Conditionals:

javascript

```javascript
let score = 85;

if (score >= 90) {
    console.log("Grade: A");
} else if (score >= 80) {
    console.log("Grade: B");
} else {
    console.log("Grade: C");
}
```

Loops:

javascript

```javascript
// For loop
for (let i = 0; i < 5; i++) {
    console.log("Number: " + i);
}

// For...of loop
let skills = ["Linux", "Git", "Docker"];
for (let skill of skills) {
    console.log(skill);
}
```

3.3 DOM Manipulation

The DOM (Document Object Model) allows JavaScript to interact with HTML elements.

Selecting Elements:

javascript

```javascript
// Select by ID
let header = document.getElementById("header");

// Select by class
let buttons = document.getElementsByClassName("btn");

// Select by tag
let paragraphs = document.getElementsByTagName("p");

// Modern selectors
let element = document.querySelector(".class-name");
let elements = document.querySelectorAll(".class-name");
```

Modifying Elements:

javascript

```javascript
// Change text content
document.getElementById("demo").textContent = "New text";

// Change HTML content
```

```javascript
document.getElementById("demo").innerHTML = "<strong>Bold text</strong>";

// Change styles
document.getElementById("demo").style.color = "red";
document.getElementById("demo").style.fontSize = "20px";

// Add/remove classes
element.classList.add("active");
element.classList.remove("hidden");
element.classList.toggle("visible");
```

Event Handling:

javascript

```javascript
// Click event
document.getElementById("myButton").addEventListener("click", function() {
    alert("Button clicked!");
});

// Input event
document.getElementById("nameInput").addEventListener("input", function(event) {
    console.log("Input value:", event.target.value);
});

// Form submit event
document.getElementById("myForm").addEventListener("submit", function(event) {
    event.preventDefault();  // Prevent default form submission
    console.log("Form submitted");
});
```

3.4 Interactive Example

Create `interactive.html`:

html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Interactive Page</title>
```

```
<style>
    body {
        font-family: Arial, sans-serif;
        max-width: 600px;
        margin: 50px auto;
        padding: 20px;
    }

    .task {
        background-color: #f0f0f0;
        padding: 10px;
        margin: 10px 0;
        border-radius: 5px;
    }

    .completed {
        text-decoration: line-through;
        background-color: #d4edda;
    }

    input[type="text"] {
        width: 70%;
        padding: 10px;
        font-size: 16px;
    }

    button {
        padding: 10px 20px;
        font-size: 16px;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }

    button:hover {
```

```
            background-color: #0056b3;
        }
    </style>
</head>
<body>
    <h1>My Task List</h1>

    <input type="text" id="taskInput" placeholder="Enter a new task">
    <button onclick="addTask()">Add Task</button>

    <div id="taskList"></div>

    <script>
        let tasks = [];

        function addTask() {
            let input = document.getElementById("taskInput");
            let taskText = input.value.trim();

            if (taskText === "") {
                alert("Please enter a task!");
                return;
            }

            tasks.push({
                text: taskText,
                completed: false
            });

            input.value = "";
            renderTasks();
        }

        function toggleTask(index) {
            tasks[index].completed = !tasks[index].completed;
            renderTasks();
        }
```

```
        function deleteTask(index) {
            tasks.splice(index, 1);
            renderTasks();
        }

        function renderTasks() {
            let taskList = document.getElementById("taskList");
            taskList.innerHTML = "";

            tasks.forEach((task, index) => {
                let taskDiv = document.createElement("div");
                taskDiv.className = "task";
                if (task.completed) {
                    taskDiv.classList.add("completed");
                }

                taskDiv.innerHTML = `
                    <span onclick="toggleTask(${index})" style="cursor: pointer">
                        ${task.text}
                    </span>
                    <button onclick="deleteTask(${index})"
                            style="float: right; background-color: #dc3545">
                        Delete
                    </button>
                `;

                taskList.appendChild(taskDiv);
            });
        }
    </script>
</body>
</html>
```

Part 4: Backend Development with Python Flask

4.1 Introduction to Flask

Flask is a lightweight Python web framework for building backend applications and APIs.

Installation:

bash

```bash
# Update package list
sudo apt update

# Install Python and pip
sudo apt install python3 python3-pip

# Install Flask
pip3 install flask
```

4.2 Your First Flask Application

Create `app.py`:

python

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello, DevOps!"

@app.route('/about')
def about():
    return "This is the about page"

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Run the application:

bash

```bash
python3 app.py
```

Visit `http://localhost:5000` in your browser.

4.3 Flask Routes and Methods

python

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# GET request
@app.route('/users', methods=['GET'])
def get_users():
    users = [
        {"id": 1, "name": "Ali"},
        {"id": 2, "name": "Sara"}
    ]
    return jsonify(users)

# GET request with parameter
@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = {"id": user_id, "name": "Ali"}
    return jsonify(user)

# POST request
@app.route('/users', methods=['POST'])
def create_user():
    data = request.get_json()
    new_user = {
        "id": 3,
        "name": data.get('name')
    }
    return jsonify(new_user), 201

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

4.4 Building a Complete API

Create `student_api.py`:

python

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory database (list of dictionaries)
students = [
    {"id": 1, "name": "Ali Ahmed", "course": "SE202", "grade": 85},
    {"id": 2, "name": "Sara Khan", "course": "SE202", "grade": 92}
]

# Get all students
@app.route('/api/students', methods=['GET'])
def get_students():
    return jsonify(students)

# Get single student
@app.route('/api/students/<int:student_id>', methods=['GET'])
def get_student(student_id):
    student = next((s for s in students if s['id'] == student_id), None)
    if student:
        return jsonify(student)
    else:
        return jsonify({"error": "Student not found"}), 404

# Create new student
@app.route('/api/students', methods=['POST'])
def create_student():
    data = request.get_json()

    new_student = {
        "id": len(students) + 1,
        "name": data.get('name'),
        "course": data.get('course'),
        "grade": data.get('grade')
    }
```

```
    students.append(new_student)
    return jsonify(new_student), 201

# Update student
@app.route('/api/students/<int:student_id>', methods=['PUT'])
def update_student(student_id):
    student = next((s for s in students if s['id'] == student_id), None)

    if not student:
        return jsonify({"error": "Student not found"}), 404

    data = request.get_json()
    student['name'] = data.get('name', student['name'])
    student['grade'] = data.get('grade', student['grade'])

    return jsonify(student)

# Delete student
@app.route('/api/students/<int:student_id>', methods=['DELETE'])
def delete_student(student_id):
    global students
    students = [s for s in students if s['id'] != student_id]
    return jsonify({"message": "Student deleted"}), 200

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

4.5 Testing the API with curl

bash

```
# Get all students
curl http://localhost:5000/api/students

# Get specific student
curl http://localhost:5000/api/students/1

# Create new student
curl -X POST http://localhost:5000/api/students \
```

```
  -H "Content-Type: application/json" \
  -d '{"name":"Ahmed Ali","course":"SE202","grade":88}'

# Update student
curl -X PUT http://localhost:5000/api/students/1 \
  -H "Content-Type: application/json" \
  -d '{"grade":95}'

# Delete student
curl -X DELETE http://localhost:5000/api/students/1
```

Part 5: Alternative Backend - Node.js Express

5.1 Introduction to Node.js and Express

Node.js allows you to run JavaScript on the server side. Express is a popular web framework for Node.js.

Installation:

bash

```
# Install Node.js and npm
sudo apt update
sudo apt install nodejs npm

# Verify installation
node --version
npm --version
```

5.2 Creating an Express Application

bash

```
# Create project directory
mkdir node-backend
cd node-backend

# Initialize npm project
npm init -y
```

```bash
# Install Express
npm install express
```

Create `server.js`:

javascript

```javascript
const express = require('express');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// Routes
app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.get('/about', (req, res) => {
  res.json({ message: 'About page', version: '1.0' });
});

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Run the server:

bash

```bash
node server.js
```

5.3 Building a Complete API with Express

Create `student_api.js`:

javascript

```javascript
const express = require('express');
const app = express();
```

```
app.use(express.json());

// In-memory database
let students = [
    { id: 1, name: "Ali Ahmed", course: "SE202", grade: 85 },
    { id: 2, name: "Sara Khan", course: "SE202", grade: 92 }
];

// Get all students
app.get('/api/students', (req, res) => {
    res.json(students);
});

// Get single student
app.get('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id === parseInt(req.params.id));
    if (!student) {
        return res.status(404).json({ error: 'Student not found' });
    }
    res.json(student);
});

// Create new student
app.post('/api/students', (req, res) => {
    const newStudent = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course,
        grade: req.body.grade
    };
    students.push(newStudent);
    res.status(201).json(newStudent);
});

// Update student
app.put('/api/students/:id', (req, res) => {
    const student = students.find(s => s.id === parseInt(req.params.id));
```

```
  if (!student) {
    return res.status(404).json({ error: 'Student not found' });
  }

  student.name = req.body.name || student.name;
  student.grade = req.body.grade || student.grade;
  res.json(student);
});

// Delete student
app.delete('/api/students/:id', (req, res) => {
  students = students.filter(s => s.id !== parseInt(req.params.id));
  res.json({ message: 'Student deleted' });
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Part 6: Connecting Frontend and Backend

6.1 CORS (Cross-Origin Resource Sharing)

When frontend and backend run on different ports, you need to enable CORS.

For Flask:

bash

```
pip3 install flask-cors
```

python

```
from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Enable CORS for all routes

# Your routes here
```

For Express:

bash

```bash
npm install cors
```

javascript

```javascript
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());  // Enable CORS
app.use(express.json());

// Your routes here
```

6.2 Complete Frontend-Backend Example

Backend (Flask) - `api_server.py`:

python

```python
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

tasks = []

@app.route('/api/tasks', methods=['GET'])
def get_tasks():
    return jsonify(tasks)

@app.route('/api/tasks', methods=['POST'])
def add_task():
    data = request.get_json()
    task = {
        "id": len(tasks) + 1,
        "text": data.get('text'),
        "completed": False
```

```
    }
    tasks.append(task)
    return jsonify(task), 201

@app.route('/api/tasks/<int:task_id>', methods=['DELETE'])
def delete_task(task_id):
    global tasks
    tasks = [t for t in tasks if t['id'] != task_id]
    return jsonify({"message": "Task deleted"})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Frontend - `frontend.html`:

html

```
<!DOCTYPE html>
<html>
<head>
    <title>Task Manager</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 600px;
            margin: 50px auto;
            padding: 20px;
        }

        .task {
            background-color: #f0f0f0;
            padding: 10px;
            margin: 10px 0;
            border-radius: 5px;
            display: flex;
            justify-content: space-between;
            align-items: center;
        }
```

```
        input[type="text"] {
            width: 70%;
            padding: 10px;
            font-size: 16px;
        }

        button {
            padding: 10px 20px;
            font-size: 16px;
            background-color: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background-color: #0056b3;
        }

        .delete-btn {
            background-color: #dc3545;
            padding: 5px 10px;
        }

        .delete-btn:hover {
            background-color: #c82333;
        }
    </style>
</head>
<body>
    <h1>Task Manager (Connected to Backend)</h1>

    <input type="text" id="taskInput" placeholder="Enter a new task">
    <button onclick="addTask()">Add Task</button>

    <div id="taskList"></div>
```

```
<script>
    const API_URL = 'http://localhost:5000/api/tasks';

    // Load tasks when page loads
    window.onload = function() {
        loadTasks();
    };

    // Load tasks from backend
    function loadTasks() {
        fetch(API_URL)
            .then(response => response.json())
            .then(tasks => {
                displayTasks(tasks);
            })
            .catch(error => {
                console.error('Error loading tasks:', error);
            });
    }

    // Add new task
    function addTask() {
        let input = document.getElementById("taskInput");
        let taskText = input.value.trim();

        if (taskText === "") {
            alert("Please enter a task!");
            return;
        }

        fetch(API_URL, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ text: taskText })
```

```
    })
    .then(response => response.json())
    .then(task => {
        input.value = "";
        loadTasks();  // Reload tasks
    })
    .catch(error => {
        console.error('Error adding task:', error);
    });
}

// Delete task
function deleteTask(taskId) {
    fetch(`${API_URL}/${taskId}`, {
        method: 'DELETE'
    })
    .then(response => response.json())
    .then(result => {
        loadTasks();  // Reload tasks
    })
    .catch(error => {
        console.error('Error deleting task:', error);
    });
}

// Display tasks in UI
function displayTasks(tasks) {
    let taskList = document.getElementById("taskList");
    taskList.innerHTML = "";

    if (tasks.length === 0) {
        taskList.innerHTML = "<p>No tasks yet. Add one above!</p>";
        return;
    }

    tasks.forEach(task => {
        let taskDiv = document.createElement("div");
```

```
            taskDiv.className = "task";

            taskDiv.innerHTML = `
                <span>${task.text}</span>
                <button class="delete-btn" onclick="deleteTask(${task.id})">Delete</button>
            `;

            taskList.appendChild(taskDiv);
        });
    }
  </script>
</body>
</html>
```

Part 7: Practical Exercises

Exercise 1: Create a Personal Portfolio Page

Create an HTML/CSS page with:

1. Your name and photo
2. A list of skills
3. A contact form
4. Styled with CSS
5. Add JavaScript to validate the form

Exercise 2: Build a Calculator API

Create a Flask or Express API with endpoints:

- `/api/add?a=5&b=3` - Addition
- `/api/subtract?a=5&b=3` - Subtraction
- `/api/multiply?a=5&b=3` - Multiplication
- `/api/divide?a=6&b=3` - Division

Exercise 3: Student Management System

Create a full-stack application: **Backend:** API with endpoints for CRUD operations on students
Frontend: HTML page with forms to add, view, update, and delete students

Exercise 4: Weather Dashboard

Create a frontend page that:

1. Has a form to enter city name
2. Displays mock weather data
3. Shows temperature, conditions, humidity
4. Styled attractively with CSS

Lab Assessment Tasks

Task 1: Personal Bio Page

Create `bio.html` with:

- Your information (name, student ID, interests)
- A styled navigation menu
- At least 3 sections with different content
- Responsive CSS styling
- A working contact form (frontend only)

Task 2: REST API for Books

Create a backend API (`books_api.py` or `books_api.js`) with:

- GET `/api/books` - List all books
- GET `/api/books/:id` - Get specific book
- POST `/api/books` - Add new book
- PUT `/api/books/:id` - Update book
- DELETE `/api/books/:id` - Delete book

Each book should have: id, title, author, year

Task 3: Interactive Quiz Page

Create `quiz.html` with:

- At least 5 multiple-choice questions about DevOps
- JavaScript to check answers
- Display score at the end
- Styled with CSS

Task 4: Todo List with Backend

Create a complete todo application:

- Backend API with Flask/Express

- Frontend HTML page
- Ability to add and delete tasks
- Tasks stored in backend
- Frontend communicates with backend via fetch API

Task 5: Documentation

Create `api-documentation.md` that:

- Explains the purpose of your API
- Lists all endpoints with examples
- Shows example requests and responses
- Explains how to run your application

Submit all files with a README explaining how to run each component.

Lab 5: Git Collaboration & GitHub

Lab Overview

This lab focuses on collaborative software development using Git and GitHub. You will learn to work with remote repositories, manage branches, handle pull requests, resolve merge conflicts, and follow best practices for team collaboration. These skills are essential for modern software development and DevOps practices.

Topics Covered:

- GitHub account setup and repository creation
- Remote repository operations (clone, pull, push)
- Branching strategies and workflows
- Pull requests and code reviews
- Merge conflict resolution
- README and documentation practices

Learning Objectives

By the end of this lab, you will be able to:

1. Create and manage repositories on GitHub
2. Clone remote repositories and push changes
3. Create and manage branches for feature development
4. Make pull requests and perform code reviews
5. Resolve merge conflicts effectively
6. Write clear README files and documentation
7. Collaborate with team members using Git workflows

Part 1: Introduction to GitHub

1.1 What is GitHub?

GitHub is a web-based platform for version control using Git. It provides a centralized place to store code, collaborate with others, and manage software projects.

Key Features:

- Remote repository hosting
- Collaboration tools (pull requests, issues)
- Code review capabilities
- Project management features
- GitHub Actions for CI/CD (covered later)
- Documentation hosting (GitHub Pages)

Git vs GitHub:

- **Git**: Version control system (software on your computer)
- **GitHub**: Cloud platform for hosting Git repositories

1.2 Creating a GitHub Account

1. Visit https://github.com
2. Click "Sign up"
3. Follow the registration process
4. Verify your email address
5. Choose the free plan

1.3 SSH Key Setup (Recommended)

SSH keys allow secure authentication without passwords.

Generate SSH Key:

bash

```bash
# Generate new SSH key
ssh-keygen -t ed25519 -C "your.email@example.com"

# Press Enter to accept default location
# Enter a passphrase (or press Enter for no passphrase)

# Start SSH agent
eval "$(ssh-agent -s)"

# Add SSH key to agent
ssh-add ~/.ssh/id_ed25519

# Display public key
cat ~/.ssh/id_ed25519.pub
```

Add to GitHub:

1. Copy the public key output
2. Go to GitHub → Settings → SSH and GPG keys
3. Click "New SSH key"
4. Paste your key and save

Test Connection:

bash

```bash
ssh -T git@github.com
```

You should see: "Hi username! You've successfully authenticated..."

Part 2: Working with Remote Repositories

2.1 Creating a Repository on GitHub

Via Web Interface:

1. Click the "+" icon → "New repository"
2. Enter repository name (e.g., `devops-project`)
3. Add description (optional)
4. Choose public or private
5. Optionally initialize with README
6. Click "Create repository"

2.2 Connecting Local and Remote Repositories

Method 1: Push an Existing Local Repository

bash

```bash
# Navigate to your local repository
cd my-project

# Initialize if not already done
git init

# Add remote origin
git remote add origin git@github.com:username/devops-project.git

# Verify remote
git remote -v

# Push to remote (first time)
git push -u origin main
```

Method 2: Clone an Existing Remote Repository

```bash
# Clone repository
git clone git@github.com:username/devops-project.git

# Navigate into cloned directory
cd devops-project

# Check remote
git remote -v
```

2.3 Basic Remote Operations

Pulling Changes:

```bash
# Fetch and merge changes from remote
git pull origin main

# Fetch changes without merging
git fetch origin

# View what was fetched
git log origin/main
```

Pushing Changes:

```bash
# Make changes to files
echo "New content" >> file.txt

# Stage and commit
git add file.txt
git commit -m "Update file content"

# Push to remote
git push origin main
```

Viewing Remote Information:

bash

```bash
# List remotes
git remote -v

# Show remote details
git remote show origin

# List remote branches
git branch -r

# List all branches (local and remote)
git branch -a
```

Part 3: Branching Strategies

3.1 Understanding Branches

Branches allow you to develop features, fix bugs, or experiment without affecting the main codebase.

Why Use Branches?

- Isolated development environment
- Parallel feature development
- Safe experimentation
- Organized workflow
- Easy rollback if needed

3.2 Creating and Managing Branches

Basic Branch Operations:

bash

```bash
# List all local branches
git branch

# List all branches including remote
git branch -a

# Create new branch
```

```bash
git branch feature-login

# Switch to branch
git checkout feature-login

# Create and switch in one command
git checkout -b feature-signup

# Modern way (Git 2.23+)
git switch feature-login
git switch -c feature-new-feature
```

Working with Branches:

```bash
# Switch to feature branch
git checkout -b feature-authentication

# Make changes on your branch
echo "# Authentication Feature" > auth.md
echo "Implementing user login" >> auth.md

git add auth.md
git commit -m "Add authentication documentation"

# Make more changes
mkdir auth/
echo "def login():" > auth/login.py

git add auth/
git commit -m "Add login function skeleton"

# Push branch to remote
git push origin feature-authentication

# Or set upstream and push
git push -u origin feature-authentication
```

Branch Management:

```bash
# Delete local branch (must not be checked out)
git branch -d feature-login

# Force delete (if not merged)
git branch -D feature-login

# Delete remote branch
git push origin --delete feature-login

# Rename current branch
git branch -m new-branch-name

# Rename another branch
git branch -m old-name new-name
```

3.3 Common Branching Strategies

Feature Branch Workflow (Recommended for this course):

```
main (production-ready code)
 |
 |-- feature-login
 |-- feature-signup
 |-- bugfix-header
 |-- feature-dashboard
```

Best Practices:

- Always create branches from updated main
- Use descriptive branch names (feature-*, bugfix-*, hotfix-*)
- Keep branches focused on single features
- Delete branches after merging

Workflow Example:

```bash
# Step 1: Start from updated main
```

```
git checkout main
git pull origin main

# Step 2: Create feature branch
git checkout -b feature-user-profile

# Step 3: Work on feature
# ... make changes ...
git add .
git commit -m "Add user profile page"

# Step 4: Keep branch updated (if main has changed)
git checkout main
git pull origin main
git checkout feature-user-profile
git merge main

# Step 5: Push feature branch
git push origin feature-user-profile

# Step 6: Create pull request on GitHub (covered next)

# Step 7: After merge, cleanup
git checkout main
git pull origin main
git branch -d feature-user-profile
```

Part 4: Pull Requests

4.1 What is a Pull Request?

A pull request (PR) is a request to merge changes from one branch into another. It's a core collaboration feature that enables:

- Code review before merging
- Discussion about changes
- Automated testing (CI/CD)
- Documentation of why changes were made

4.2 Creating a Pull Request

Step-by-Step Process:

1. **Push your branch to GitHub:**

bash

```bash
git push origin feature-user-profile
```

2. **Create PR on GitHub:**
   - Go to your repository on GitHub
   - Click "Pull requests" tab
   - Click "New pull request"
   - Select base branch (usually `main`) and compare branch (your feature branch)
   - Review changes
   - Click "Create pull request"
   - Add title and description
   - Click "Create pull request"

Good PR Description Template:

markdown

```markdown
## Description
Brief description of what this PR does

## Changes Made
- Added user profile page
- Created profile API endpoint
- Updated navigation menu

## Testing
- Tested on local environment
- All existing tests pass

## Screenshots (if applicable)
[Add screenshots here]

## Related Issues
Fixes #123
```

4.3 Code Review Process

As a Reviewer:

1. Go to the "Pull requests" tab
2. Click on the PR to review
3. Click "Files changed" tab
4. Review code changes
5. Add comments by clicking the "+" next to line numbers
6. Click "Review changes" button
7. Choose: Approve, Request changes, or Comment
8. Submit review

Responding to Review Comments:

bash

```bash
# Make requested changes
# ... edit files ...

# Commit changes
git add .
git commit -m "Address review comments: fix variable naming"

# Push to same branch (PR updates automatically)
git push origin feature-user-profile
```

4.4 Merging Pull Requests

Three Merge Strategies:

1. **Merge Commit** (Default)
   - Creates a merge commit
   - Preserves all individual commits
   - Full history retained
2. **Squash and Merge**
   - Combines all commits into one
   - Cleaner history
   - Good for feature branches with many small commits
3. **Rebase and Merge**
   - Applies commits on top of base branch
   - Linear history
   - No merge commit

After Merging:

bash

```bash
# Update local main branch
git checkout main
git pull origin main

# Delete feature branch locally
git branch -d feature-user-profile

# Delete feature branch on remote (if not auto-deleted)
git push origin --delete feature-user-profile
```

Part 5: Merge Conflicts

5.1 Understanding Merge Conflicts

Conflicts occur when Git can't automatically merge changes because:

- Same lines were modified in both branches
- File was modified in one branch and deleted in another
- Both branches made incompatible changes

5.2 Creating a Conflict (Practice)

bash

```bash
# Setup: Create a repository
mkdir conflict-demo
cd conflict-demo
git init

# Create initial file
echo "Line 1: Original" > file.txt
echo "Line 2: Original" >> file.txt
git add file.txt
git commit -m "Initial commit"

# Create and switch to branch A
git checkout -b branch-a
echo "Line 1: Modified by Branch A" > file.txt
echo "Line 2: Original" >> file.txt
git add file.txt
```

```bash
git commit -m "Branch A changes"

# Switch to main and create branch B
git checkout main
git checkout -b branch-b
echo "Line 1: Modified by Branch B" > file.txt
echo "Line 2: Original" >> file.txt
git add file.txt
git commit -m "Branch B changes"

# Try to merge branch-a into branch-b
git merge branch-a
# CONFLICT!
```

5.3 Resolving Conflicts

Step 1: Identify Conflicted Files

```bash
# Git will show which files have conflicts
git status
```

Step 2: Open Conflicted File

The file will contain conflict markers:

```
<<<<<<< HEAD
Line 1: Modified by Branch B
=======
Line 1: Modified by Branch A
>>>>>>> branch-a
Line 2: Original
```

Explanation:

- `<<<<<<< HEAD`: Start of your current branch's changes
- `=======`: Separator
- `>>>>>>> branch-a`: End of incoming branch's changes

Step 3: Resolve the Conflict

Edit the file to choose what to keep:

```bash
# Option 1: Keep branch B version
Line 1: Modified by Branch B
Line 2: Original

# Option 2: Keep branch A version
Line 1: Modified by Branch A
Line 2: Original

# Option 3: Keep both (merge manually)
Line 1: Modified by Branch A and Branch B
Line 2: Original

# Option 4: Write new content
Line 1: New combined version
Line 2: Original
```

Step 4: Mark as Resolved

```bash
# After editing, stage the resolved file
git add file.txt

# Check status
git status

# Complete the merge
git commit -m "Resolve merge conflict between branch-a and branch-b"
```

5.4 Aborting a Merge

If you want to cancel the merge:

```bash
# Abort the merge and return to pre-merge state
git merge --abort
```

5.5 Preventing Conflicts

Best Practices:

1. Pull latest changes before starting work
2. Commit and push frequently
3. Keep branches short-lived
4. Communicate with team about what you're working on
5. Merge main into your branch regularly

```bash
# Regular workflow to stay updated
git checkout main
git pull origin main
git checkout your-feature-branch
git merge main
# Resolve any conflicts now, not later
```

Part 6: Documentation Best Practices

6.1 Writing a Good README

A README is often the first thing people see in your repository. It should explain what the project does and how to use it.

Basic README Template:

```markdown
# Project Title

Brief description of what this project does

## Features

- Feature 1
- Feature 2
- Feature 3

## Installation
```bash
```
```

```bash
# Clone the repository
git clone https://github.com/username/project-name.git

# Navigate to directory
cd project-name

# Install dependencies
npm install
```

Usage

```bash
# Run the application
npm start
```

Examples

```bash
# Example command
./script.sh --option value
```

Requirements

- Python 3.8+
- Node.js 14+
- Git

Contributing

1. Fork the repository
2. Create your feature branch (`git checkout -b feature/AmazingFeature`)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

License

This project is licensed under the MIT License

Authors

- Your Name - Initial work

Acknowledgments

- Hat tip to anyone whose code was used
- Inspiration
- References

6.2 Other Important Documentation Files

CONTRIBUTING.md:

markdown

Contributing Guidelines

How to Contribute

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Write or update tests
5. Ensure all tests pass
6. Submit a pull request

Code Style

- Follow PEP 8 for Python code
- Use meaningful variable names
- Add comments for complex logic

Commit Messages

- Use present tense ("Add feature" not "Added feature")

- Be descriptive but concise
- Reference issues when applicable

.gitignore:

```bash
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/

# Node
node_modules/
npm-debug.log
yarn-error.log

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db

# Logs
*.log

# Environment variables
.env
.env.local
```

Part 7: Practical Exercises

Exercise 1: First GitHub Repository

Tasks:

1. Create a new repository on GitHub called `git-practice`
2. Clone it to your local machine
3. Create a README.md with project description
4. Add a Python script that prints "Hello, DevOps!"
5. Create a .gitignore file
6. Commit and push changes
7. Verify changes appear on GitHub

bash

```bash
# Solution
git clone git@github.com:yourusername/git-practice.git
cd git-practice

echo "# Git Practice Repository" > README.md
echo "This repository is for practicing Git and GitHub" >> README.md

echo "print('Hello, DevOps!')" > hello.py

cat > .gitignore << EOF
*.pyc
__pycache__/
.vscode/
EOF

git add .
git commit -m "Initial commit: Add README, Python script, and gitignore"
git push origin main
```

Exercise 2: Feature Branch Workflow

Tasks:

1. Create a new branch called `feature-calculator`
2. Add a calculator.py file with add and subtract functions
3. Commit your changes
4. Push the branch to GitHub
5. Create a pull request
6. Merge the pull request

7. Delete the branch

bash

```bash
# Solution
git checkout -b feature-calculator

cat > calculator.py << EOF
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero"
    return a / b

if __name__ == "__main__":
    print("Calculator Functions")
    print("5 + 3 =", add(5, 3))
    print("5 - 3 =", subtract(5, 3))
    print("5 * 3 =", multiply(5, 3))
    print("6 / 3 =", divide(6, 3))
EOF

git add calculator.py
git commit -m "Add calculator with basic operations"
git push origin feature-calculator

# Now create PR on GitHub, merge it, then:
git checkout main
git pull origin main
git branch -d feature-calculator
```

Exercise 3: Collaborative Workflow (Pair Exercise)

Scenario: Work with a partner on the same repository

Partner A Tasks:

1. Create repository and add Partner B as collaborator
2. Create initial README
3. Push to GitHub

Partner B Tasks:

1. Clone the repository
2. Create branch `feature-docs`
3. Add CONTRIBUTING.md
4. Push and create PR

Partner A Tasks:

1. Review Partner B's PR
2. Add comments
3. Approve and merge

Partner B Tasks:

1. Create branch `feature-license`
2. Add LICENSE file
3. Push and create PR

Partner A Tasks:

1. Merge the PR

bash

```bash
# Partner A
git init collaborative-project
cd collaborative-project
echo "# Collaborative Project" > README.md
git add README.md
git commit -m "Initial commit"
git remote add origin git@github.com:partnera/collaborative-project.git
git push -u origin main

# Partner B
```

```bash
git clone git@github.com:partnera/collaborative-project.git
cd collaborative-project
git checkout -b feature-docs
# Create CONTRIBUTING.md as shown earlier
git add CONTRIBUTING.md
git commit -m "Add contributing guidelines"
git push origin feature-docs
# Create PR on GitHub
```

Exercise 4: Merge Conflict Resolution

Tasks:

1. Create a branch called `branch-1`
2. Modify line 1 of README.md
3. Commit and switch back to main
4. Create branch called `branch-2`
5. Modify the same line differently
6. Commit changes
7. Try to merge `branch-1` into `branch-2`
8. Resolve the conflict
9. Complete the merge

bash

```bash
# Solution
git checkout -b branch-1
echo "# Modified by Branch 1" > README.md
git add README.md
git commit -m "Modify README in branch-1"

git checkout main
git checkout -b branch-2
echo "# Modified by Branch 2" > README.md
git add README.md
git commit -m "Modify README in branch-2"

git merge branch-1
# Conflict occurs!

# Edit README.md to resolve conflict
```

```bash
echo "# Modified by Both Branches" > README.md

git add README.md
git commit -m "Resolve merge conflict between branch-1 and branch-2"
```

Exercise 5: GitHub Repository Setup

Tasks:

Create a complete repository for a DevOps project with:

1. Proper README with all sections
2. .gitignore for Python
3. CONTRIBUTING.md
4. Sample Python application
5. At least 3 meaningful commits
6. At least 1 merged pull request

Part 8: Advanced Git Topics

8.1 Git Tags

Tags mark specific points in history, typically for releases.

```bash
# Create a lightweight tag
git tag v1.0.0

# Create an annotated tag (recommended)
git tag -a v1.0.0 -m "Release version 1.0.0"

# List all tags
git tag

# Show tag details
git show v1.0.0

# Push tags to remote
git push origin v1.0.0

# Push all tags
```

```bash
git push origin --tags

# Delete local tag
git tag -d v1.0.0

# Delete remote tag
git push origin --delete v1.0.0
```

8.2 Git Stash

Temporarily save changes without committing.

```bash
# Stash current changes
git stash

# Stash with message
git stash save "Work in progress on feature X"

# List all stashes
git stash list

# Apply most recent stash
git stash apply

# Apply and remove stash
git stash pop

# Apply specific stash
git stash apply stash@{1}

# Delete stash
git stash drop stash@{0}

# Delete all stashes
git stash clear
```

Use Case:

```bash
# You're working on a feature
echo "Incomplete feature" > feature.txt
git add feature.txt

# Suddenly need to fix a bug
git stash

# Fix bug
git checkout -b hotfix-bug
# ... fix bug ...
git add .
git commit -m "Fix critical bug"

# Return to feature work
git checkout feature-branch
git stash pop
# Continue working on feature
```

8.3 Viewing History

```bash
# View commit history
git log

# One line per commit
git log --oneline

# Graphical representation
git log --graph --oneline --all

# Show commits by author
git log --author="Your Name"

# Show commits in date range
git log --since="2 weeks ago"
git log --until="2026-01-20"
```

```bash
# Show commits affecting specific file
git log -- filename.txt

# Show commit statistics
git log --stat

# Search commit messages
git log --grep="bug fix"
```

8.4 Undoing Changes

```bash
# Discard changes in working directory
git checkout -- filename.txt

# Unstage file (keep changes)
git reset HEAD filename.txt

# Undo last commit (keep changes)
git reset --soft HEAD~1

# Undo last commit (discard changes)
git reset --hard HEAD~1

# Amend last commit message
git commit --amend -m "New commit message"

# Amend last commit (add forgotten files)
git add forgotten-file.txt
git commit --amend --no-edit

# Revert a commit (creates new commit)
git revert commit-hash
```

Lab Assessment Tasks

Task 1: Repository Setup

Create a GitHub repository called `devops-portfolio` with:

1. Comprehensive README.md including:
    - Project description
    - Installation instructions
    - Usage examples
    - Requirements
    - Contributing guidelines
2. .gitignore for your chosen language
3. LICENSE file (MIT or Apache 2.0)
4. Initial commit with meaningful message

Task 2: Feature Development

Using the repository from Task 1:

1. Create branch `feature-about-page`
2. Add an `about.html` file with your information
3. Commit with meaningful message
4. Push branch to GitHub
5. Create pull request with detailed description
6. Merge pull request
7. Delete feature branch (local and remote)
8. Update local main branch

Document all commands used.

Task 3: Collaboration Simulation

Working alone or with a partner:

1. Create branch `feature-scripts`
2. Add 3 different shell scripts
3. Make separate commits for each script
4. Push branch
5. Create branch `feature-docs` from `feature-scripts`
6. Add documentation for the scripts
7. Create two separate pull requests
8. Merge them in logical order

Task 4: Conflict Resolution

1. Create branch `version-a` from main
2. Modify README.md (change line 1)
3. Commit changes
4. Return to main, create branch `version-b`
5. Modify same line in README.md differently

6. Commit changes
7. Attempt to merge `version-a` into `version-b`
8. Resolve conflict
9. Complete merge
10. Document the conflict and resolution process

Task 5: Complete Workflow

Create a mini-project repository demonstrating:

1. At least 5 meaningful commits on main
2. At least 3 feature branches
3. At least 2 merged pull requests
4. At least 1 resolved merge conflict
5. Proper documentation (README, CONTRIBUTING)
6. Use of .gitignore
7. At least 1 tag for a "release"

Submit:

- Repository URL
- Document with screenshots of:
    - Pull requests
    - Merge conflict resolution
    - Commit history
    - Branch structure